Lab 4B - COVID19 Regression

In part 2 of this lab we will use multi-layer perceptrons (also called fully connected neural networks) in the context of regression to predict COVID19 cases.

Created By: Tuan Do & Bernie Boscoe

Latest Update: Tuan Do

In [87]:
```
## Code snippet to wrap the lines when converting to PDFs
from IPython.display import HTML, display

display(HTML("""
<style>
pre, code {
  white-space: pre-wrap !important;
  word-wrap: break-word !important;
  overflow-wrap: anywhere !important;
}
.jp-RenderedHTMLCommon pre {
  white-space: pre-wrap !important;
}
</style>
"""))
```

Step 1. Big Picture

For this task, we will be looking at positive Covid test rates all over the world, from Jan 2020- Aug 2020. Incidentally this is a very popular set used on Kaggle (a machine learning competition site) in inumerable ways. Our goals are as follows:

Predict COVID19 cases as a function of time and location on Earth.

We will try to model a slice of our dataset using a fully connected neural network. Our metrics will include: Mean squared error (MSE) to assess training vs test sets, and finding the optimal neural network weights for regression. We will then trying to assess if we've overfit, or underfit, and possible remedies. (If, during the lab, you are wondering 'why are we doing it this way?' Know that, this is an actual job interview assessment in "the real world". And, of course if you would like to propose a 'better way' to attack the problem (what order, lowest MSE, or, another type of fit, that would be a great discussion topic as well).

So, keep it simple, there's not too much coding or work that goes into this task. Good luck!

Step 2. Get the Data

The data you will use here are publicly available world wide COVID cases from Jan. 2020 to Aug. 2020.

In [1]:
```
import numpy.matlib as matlib
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# increase font size
plt.rcParams.update({'font.size': 16})

# load the time_series_covid_19_confirmed.csv using pandas in your lab folder.
data = pd.read_csv('time_series_covid_19_confirmed.csv')


# PUT YOUR ANSWER HERE
```

In [2]:
```
data
```

Out[2]:

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/2
0	NaN	Afghanistan	33.939110	67.709953	0	0	
1	NaN	Albania	41.153300	20.168300	0	0	
2	NaN	Algeria	28.033900	1.659600	0	0	
3	NaN	Andorra	42.506300	1.521800	0	0	
4	NaN	Angola	-11.202700	17.873900	0	0	
...	...	...	...	...	...	...	
261	NaN	West Bank and Gaza	31.952200	35.233200	0	0	
262	NaN	Western Sahara	24.215500	-12.885800	0	0	
263	NaN	Yemen	15.552727	48.516388	0	0	
264	NaN	Zambia	-13.133897	27.849332	0	0	
265	NaN	Zimbabwe	-19.015438	29.154857	0	0	

266 rows × 225 columns

Step 3. Exploring the Data

Question 1

(5 pts)

Check the shape, and head, plot, visualize, and have a look at the data with whatever ways you think will help your task. Comment throughout on observations. Note: There are NaNs (hypothesize why these occur in the dataset).

In [3]:
```
# Check the shape
print(f"Shape: {data.shape}")
# 266 rows of countries and provinces (when available) and 225 columns of
covid data
```

```
Shape: (266, 225)
```

In [4]:
```
# to see the first few rows
data.head()
```

Out[4]:

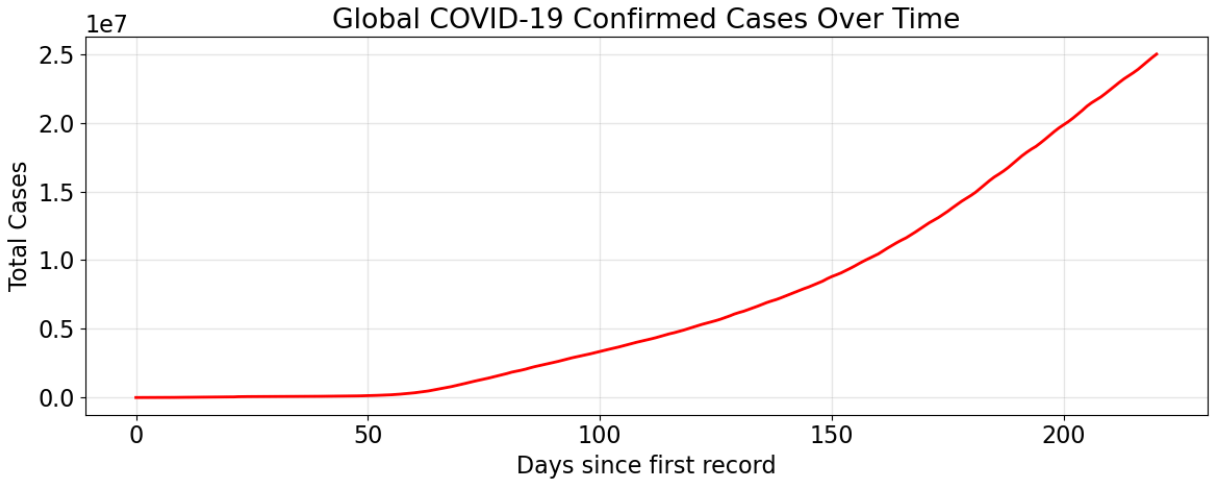
	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	
1	NaN	Albania	41.15330	20.168300	0	0	0	
2	NaN	Algeria	28.03390	1.659600	0	0	0	
3	NaN	Andorra	42.50630	1.521800	0	0	0	
4	NaN	Angola	-11.20270	17.873900	0	0	0	

5 rows × 225 columns

Global visualization of cases

In [5]:
```
# A basic visualization of the total number of cases with time elapsed
since first recorded -- we can see that the number of cases rise
exponentially with time
date_cols = data.columns[4:]
global_cases = data[date_cols].sum(axis=0)

plt.figure(figsize=(12, 5))
plt.plot(global_cases.values, linewidth=2, color='red')
plt.title('Global COVID-19 Confirmed Cases Over Time')
plt.xlabel('Days since first record')
plt.ylabel('Total Cases')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



There are 266 rows each corresponding to a country and in certain cases a specific province in the chosen country. As a time series, the number of columns are 225, each column corresponding to the covid cases at a specific date. A clear reason, why we have NaNs in the first column for instance is the fact that provincial data is not available for all countries. Examples of countries with specialised provincial data available is Canada and China.

In [6]:
```
data['Country/Region']
```

Out[6]:
```
0             Afghanistan
1                 Albania
2                 Algeria
3                 Andorra
4                  Angola
               ...
261    West Bank and Gaza
262        Western Sahara
263                 Yemen
264                Zambia
265              Zimbabwe
Name: Country/Region, Length: 266, dtype: object
```

Compare the covid data for two specific highly populated countries eg. China and India

In [7]:
```
date_columns = data.columns[4:]

# data for China (summing all the provinces)
china_data = data[data['Country/Region'] == 'China']
[date_columns].sum(axis=0)

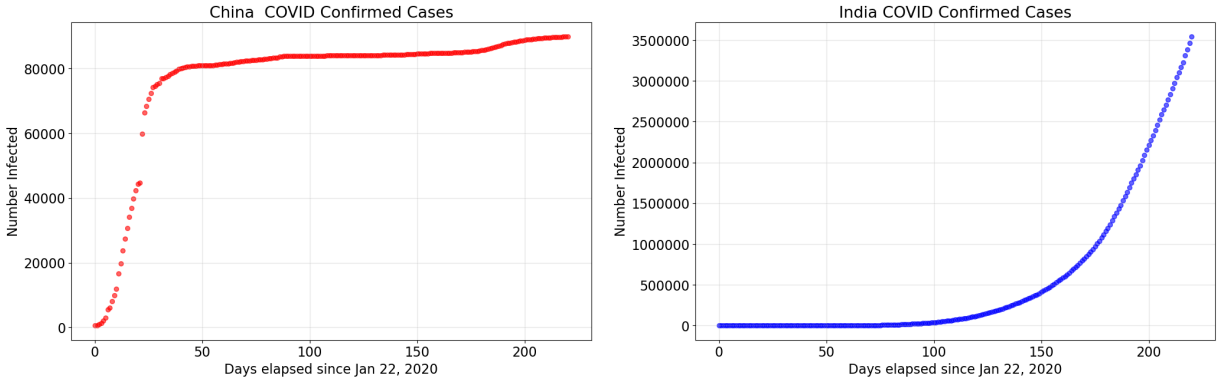
# data for India (country level data)
india_data = data[data['Country/Region'] == 'India']
[date_columns].sum(axis=0)
```

```
days_elapsed = np.arange(len(date_columns))
```

In [8]:
```
fig, axes = plt.subplots(1, 2, figsize=(25, 7))

#  Scatter plot - China
axes[0].scatter(days_elapsed, china_data.values, color='red', alpha=0.6,
s=30)
axes[0].set_xlabel('Days elapsed since Jan 22, 2020')
axes[0].set_ylabel('Number Infected')
axes[0].set_title('China  COVID Confirmed Cases')
axes[0].grid(True, alpha=0.3)
axes[0].ticklabel_format(style='plain', axis='y')

# Scatter plot - India
axes[1].scatter(days_elapsed, india_data.values, color='blue', alpha=0.6,
s=30)
axes[ 1].set_xlabel('Days elapsed since Jan 22, 2020')
axes[ 1].set_ylabel('Number Infected')
axes[ 1].set_title('India COVID Confirmed Cases')
axes[ 1].grid(True, alpha=0.3)
axes[ 1].ticklabel_format(style='plain', axis='y')
```



It is interesting to note how different the progression of COVID was with these two countries. While the cases sky rocketed for China intially, it almost thermalized with time. India on the other had started slow and then the cases started increasing exponentially due to the high population density and lack of infrastrcuture to control the spread

Below I use Claude to play around with the data a bit ! The prompt used was "Can we do something cool like get a heat map for number of cases as a function of latitude and longitude and plot the number of cases on the worldmap " ? I think plot 1 is kinda useless since the bubbles are humongous but the other visualizations are pretty useful !

In [9]:
```
# Claude used for this visualization - the prompt used is mentioned above !

# Get all date columns (columns after the first 4 metadata columns)
date_columns = data.columns[4:]

# Get latitude and longitude for all rows
lat = data['Lat'].values
long = data['Long'].values
```

```
countries = data['Country/Region'].values

# Get total cases for each location (sum across all time points)
total_cases = data[date_columns].sum(axis=1).values

# Get cases at the last time point (most recent)
latest_cases = data[date_columns[-1]].values

print(f"\nLatitude range: {lat.min():.2f} to {lat.max():.2f}")
print(f"Longitude range: {long.min():.2f} to {long.max():.2f}")
print(f"Total cases range: {total_cases.min():,.0f} to
{total_cases.max():,.0f}")

#
==========================================================================
=
# VISUALIZATION 1: Scatter plot on world map with bubble sizes
#
==========================================================================
=

fig, axes = plt.subplots(2, 2, figsize=(20, 12))

# Plot 1: Total cases (bubble size and color)
scatter1 = axes[0, 0].scatter(long, lat, s=total_cases/1000, c=total_cases,
                              cmap='Reds', alpha=0.6, edgecolors='black',
linewidth=0.5)
axes[0, 0].set_xlabel('Longitude')
axes[0, 0].set_ylabel('Latitude')
axes[0, 0].set_title('COVID-19 Total Cases by Location\n(Bubble size =
Total cases)',
                     fontweight='bold', pad=15)
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].set_xlim(-180, 180)
axes[0, 0].set_ylim(-90, 90)
cbar1 = plt.colorbar(scatter1, ax=axes[0, 0])
cbar1.set_label('Total Cases', rotation=270, labelpad=20)

# Add world coastlines (simple approximation)
axes[0, 0].axhline(y=0, color='gray', linestyle='--', alpha=0.3,
linewidth=0.5)
axes[0, 0].axvline(x=0, color='gray', linestyle='--', alpha=0.3,
linewidth=0.5)

# Plot 2: Latest cases (most recent day)
scatter2 = axes[0, 1].scatter(long, lat, s=latest_cases/1000,
c=latest_cases,
                              cmap='YlOrRd', alpha=0.6, edgecolors='black',
linewidth=0.5)
axes[0, 1].set_xlabel('Longitude')
axes[0, 1].set_ylabel('Latitude')
axes[0, 1].set_title('COVID-19 Latest Day Cases by Location\n(Bubble size =
Latest cases)',
                     fontweight='bold', pad=15)
axes[0, 1].grid(True, alpha=0.3)
axes[0, 1].set_xlim(-180, 180)
```

```
axes[0, 1].set_ylim(-90, 90)
cbar2 = plt.colorbar(scatter2, ax=axes[0, 1])
cbar2.set_label('Latest Cases', rotation=270, labelpad=20)

# Add world coastlines
axes[0, 1].axhline(y=0, color='gray', linestyle='--', alpha=0.3,
linewidth=0.5)
axes[0, 1].axvline(x=0, color='gray', linestyle='--', alpha=0.3,
linewidth=0.5)

# Plot 3: Heatmap using 2D histogram
heatmap, xedges, yedges = np.histogram2d(long, lat, bins=30,
weights=total_cases)
extent = [xedges[0], xedges[-1], yedges[0], yedges[-1]]
im3 = axes[1, 0].imshow(heatmap.T, extent=extent, origin='lower',
cmap='hot',
                        aspect='auto', interpolation='bilinear', alpha=0.8)
axes[1, 0].set_xlabel('Longitude')
axes[1, 0].set_ylabel('Latitude')
axes[1, 0].set_title('COVID-19 Heatmap (2D Density)\nWarmer colors = More
cases',
                     fontweight='bold', pad=15)
axes[1, 0].grid(True, alpha=0.3)
cbar3 = plt.colorbar(im3, ax=axes[1, 0])
cbar3.set_label('Case Density', rotation=270, labelpad=20)

# Plot 4: Log scale for better visualization
log_cases = np.log10(total_cases + 1)  # +1 to avoid log(0)
scatter4 = axes[1, 1].scatter(long, lat, s=100, c=log_cases,
                              cmap='plasma', alpha=0.7, edgecolors='black',
linewidth=0.5)
axes[1, 1].set_xlabel('Longitude')
axes[1, 1].set_ylabel('Latitude')
axes[1, 1].set_title('COVID-19 Cases by Location (Log Scale)\nBetter color
distribution',
                     fontweight='bold', pad=15)
axes[1, 1].grid(True, alpha=0.3)
axes[1, 1].set_xlim(-180, 180)
axes[1, 1].set_ylim(-90, 90)
cbar4 = plt.colorbar(scatter4, ax=axes[1, 1])
cbar4.set_label('Log10(Cases)', rotation=270, labelpad=20)

# Add world coastlines
axes[1, 1].axhline(y=0, color='gray', linestyle='--', alpha=0.3,
linewidth=0.5)
axes[1, 1].axvline(x=0, color='gray', linestyle='--', alpha=0.3,
linewidth=0.5)

plt.tight_layout()
plt.savefig('covid_world_heatmap.png', dpi=300, bbox_inches='tight')
print("\nWorld map heatmap saved as 'covid_world_heatmap.png'")
plt.show()

#
==========================================================================
=
```

```
# VISUALIZATION 2: Interactive-style plot with country labels
#
==========================================================================
=

fig, ax = plt.subplots(figsize=(18, 10))

# Create scatter plot
scatter = ax.scatter(long, lat, s=total_cases/500, c=total_cases,
                     cmap='Reds', alpha=0.6, edgecolors='black',
linewidth=1)

# Add labels for top 10 countries by cases
top_10_indices = np.argsort(total_cases)[-10:]
for idx in top_10_indices:
    ax.annotate(countries[idx],
                xy=(long[idx], lat[idx]),
                xytext=(5, 5),
                textcoords='offset points',
                fontsize=10,
                bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow',
alpha=0.7),
                arrowprops=dict(arrowstyle='->',
connectionstyle='arc3,rad=0'))

ax.set_xlabel('Longitude', fontsize=14)
ax.set_ylabel('Latitude', fontsize=14)
ax.set_title('COVID-19 World Map with Top 10 Countries Labeled\n(Bubble
size proportional to total cases)',
             fontweight='bold', fontsize=16, pad=20)
ax.grid(True, alpha=0.3)
ax.set_xlim(-180, 180)
ax.set_ylim(-90, 90)

# Add colorbar
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Total Cases', rotation=270, labelpad=25, fontsize=12)

# Add continent boundaries (approximate)
ax.axhline(y=0, color='gray', linestyle='--', alpha=0.3, linewidth=1)
ax.axvline(x=0, color='gray', linestyle='--', alpha=0.3, linewidth=1)

plt.tight_layout()
plt.savefig('covid_world_map_labeled.png', dpi=300, bbox_inches='tight')
print("Labeled world map saved as 'covid_world_map_labeled.png'")
plt.show()

#
==========================================================================
=
# STATISTICS
#
==========================================================================
=

print("\n" + "=" * 80)
```

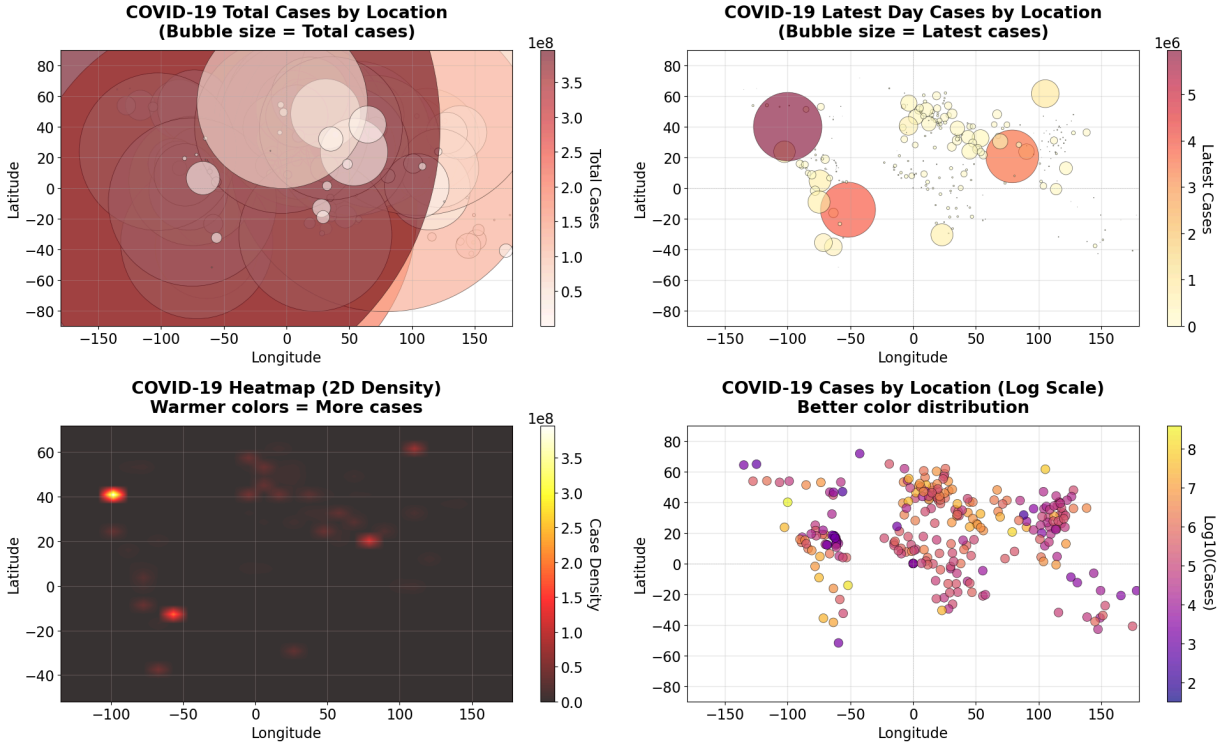
```
print("GEOGRAPHIC DISTRIBUTION STATISTICS")
print("=" * 80)

# Find hotspots
print("\nTop 10 locations by total cases:")
top_10 = np.argsort(total_cases)[-10:][::-1]
for i, idx in enumerate(top_10, 1):
    print(f"{i:2d}. {countries[idx]:20s} - {total_cases[idx]:>12,.0f} cases
"
          f"(Lat: {lat[idx]:>7.2f}, Long: {long[idx]:>7.2f})")

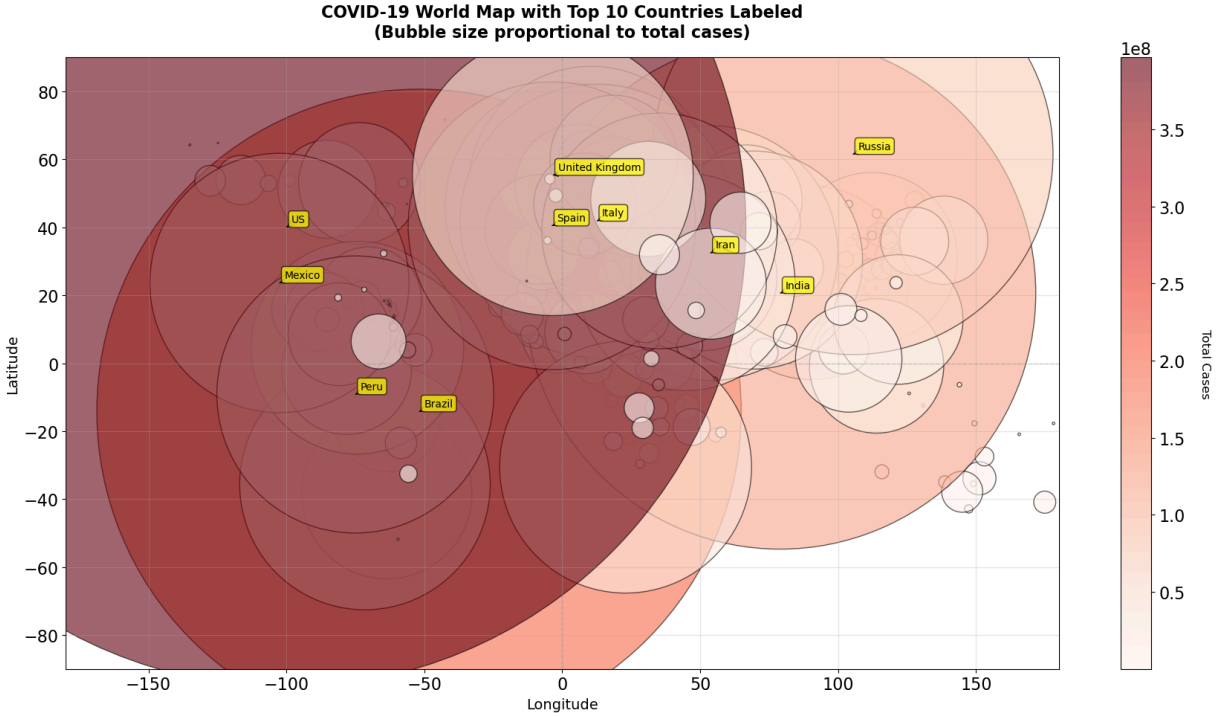
print("\n" + "=" * 80)
print("KEY OBSERVATIONS:")
print("=" * 80)
print("- Bubble sizes are proportional to total cases")
print("- Color intensity shows case concentration")
print("- Heatmap reveals geographic clustering of outbreaks")
print("- Log scale helps visualize the full range of case numbers")
print("=" * 80)
```

```
Latitude range: -51.80 to 71.71
Longitude range: -135.00 to 178.06
Total cases range: 32 to 396,509,801

World map heatmap saved as 'covid_world_heatmap.png'
```




```
Labeled world map saved as 'covid_world_map_labeled.png'
```




```
================================================================================
====
GEOGRAPHIC DISTRIBUTION STATISTICS
================================================================================
====

Top 10 locations by total cases:
 1. US                  -  396,509,801 cases (Lat:   40.00, Long: -100.00)
 2. Brazil              -  194,966,323 cases (Lat:  -14.23, Long:  -51.93)
 3. India               -  122,750,211 cases (Lat:   20.59, Long:   78.96)
 4. Russia              -   75,257,606 cases (Lat:   61.52, Long:  105.32)
 5. Spain               -   38,804,508 cases (Lat:   40.46, Long:   -3.75)
 6. United Kingdom      -   37,004,970 cases (Lat:   55.38, Long:   -3.44)
 7. Peru                -   35,986,385 cases (Lat:   -9.19, Long:  -75.02)
 8. Italy               -   35,115,294 cases (Lat:   41.87, Long:   12.57)
 9. Mexico              -   31,510,789 cases (Lat:   23.63, Long: -102.55)
10. Iran                -   30,414,143 cases (Lat:   32.43, Long:   53.69)

================================================================================
====
KEY OBSERVATIONS:
================================================================================
====
- Bubble sizes are proportional to total cases
- Color intensity shows case concentration
- Heatmap reveals geographic clustering of outbreaks
- Log scale helps visualize the full range of case numbers
================================================================================
====
```

4. Prepare Data

This step is mostly done for you. But- I highly recommend tracing through this so you understand this common data manipulation. Notice Pandas is being used to examine the data, but we will transform data into different arrays and don't need to think about Pandas when we are analyzing.

In [10]:
```
# let's only use 50 rows and 100 columns, just to make this easier (and
faster)
rows=50
cols=100
data_new=data.iloc[0:rows,4:cols+4]
print(data_new.shape)
```

```
(50, 100)
```

In [12]:
```
data_new.head()
```

Out[12]:

	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	1/28/20	1/29/20	1/30/20	1/
0	0	0	0	0	0	0	0	0	0	
1	0	0	0	0	0	0	0	0	0	
2	0	0	0	0	0	0	0	0	0	
3	0	0	0	0	0	0	0	0	0	
4	0	0	0	0	0	0	0	0	0	

5 rows × 100 columns

In [11]:
```
# Now we are going to convert the Tablular data to format {X,Y}, where X=
{Longitude, Latitude, Date}, Y={#infected}
#Finding the date indices

data_row=data_new.sum(axis=0) # bb per day summing up positive cases into
cols
days=range(0,data_row.shape[0])
days_mat=matlib.repmat(np.array(days),data_new.shape[0],1)
print(days_mat.shape)
```

```
(50, 100)
```

In [13]:
```
#Lets create data X-{X1,X2,X3}, where X1=lat, X2=long, X3=date, Y=#affected
X=np.zeros((days_mat.shape[0]*days_mat.shape[1],3))
Y=np.zeros((days_mat.shape[0]*days_mat.shape[1],1))
lat_long=np.array(data.iloc[:,2:4])
data_new=np.array(data_new)
for r in range(days_mat.shape[0]): #all locations

X[r*days_mat.shape[1]:r*days_mat.shape[1]+days_mat.shape[1],0]=lat_long[r,0
]*np.ones((days_mat.shape[1],)) #setting Latitude

X[r*days_mat.shape[1]:r*days_mat.shape[1]+days_mat.shape[1],1]=lat_long[r,1
]*np.ones((days_mat.shape[1],)) #setting Longitude
```

```
X[r*days_mat.shape[1]:r*days_mat.shape[1]+days_mat.shape[1],2]=np.reshape(d
ays,(days_mat.shape[1],)) #setting the date

Y[r*days_mat.shape[1]:r*days_mat.shape[1]+days_mat.shape[1]]=np.reshape(dat
a_new[r,:],((days_mat.shape[1],1)))
```

In [14]:
```
print(np.shape(X))
print(np.shape(Y))
#Data Preparation is Done!

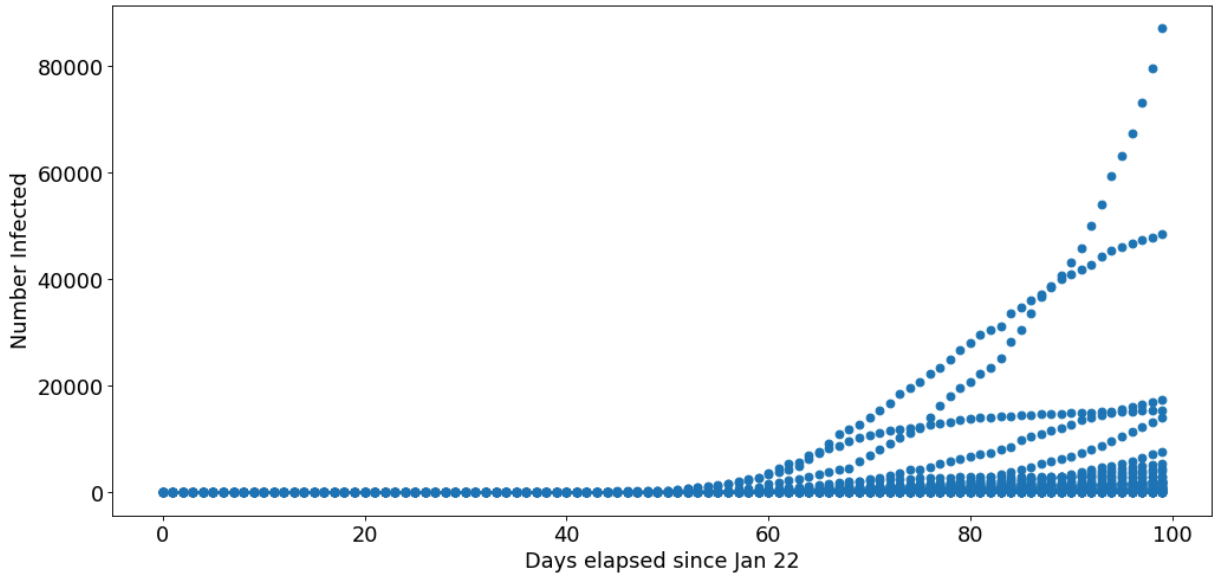
# Check nans
X[np.isnan(X)]
```

```
(5000, 3)
(5000, 1)
```

Out[14]:
```
array([], dtype=float64)
```

In [55]:
```
# Let's plot the data in this new format
plt.figure(figsize=(15, 7), dpi=80)
plt.scatter(X[:,2],Y[:])
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
print(np.shape(X))
```

```
(5000, 3)
```



5. Select a Model and Train

Here, we will use a multi-layer perceptron (MLP), also called a feed-forward neural network (FNN), or fully-connected neural network for regression. A fully connected neural network is a network where every node in one layer is connected to every node in the next layer. We will just multiple hidden layers to create a network that can learn non-linear relationships between the input and output.

In [37]:
```
# First, let's split the training and test data. If you are wondering why
we've gone back to just test and train, it is merely to mimic the actual
job interview assignment. (In general, train, test, val is better, this is
faster)

from sklearn.model_selection import train_test_split
#Split data to train and test
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
random_state=10)
```

Building a multi-layer perceptron (MLP) for regression with keras and pytorch

In this section, we will learn to use keras (https://keras.io/) and pytorch (https://pytorch.org/), two of the most popular deep learning frameworks, to build and train a neural network for regression. Keras is a high-level wrapper around different types of lower level frameworks such as pytorch, tensorflow, and JAX. The advantage of keras is that it is a common framework so that you can convert to using pytorch or tensorflow easily if you want to. In our textbook (Hands-On Machine Learning), it uses keras with the tensorflow backend, but most of the code will apply for the pytorch backend as well.

In [15]:
```
pip install keras
```

```
Requirement already satisfied: keras in /Users/ameygaikwad/anaconda3/lib/pyt
hon3.11/site-packages (3.13.2)
Requirement already satisfied: absl-py in /Users/ameygaikwad/anaconda3/lib/p
ython3.11/site-packages (from keras) (2.4.0)
Requirement already satisfied: numpy in /Users/ameygaikwad/anaconda3/lib/pyt
hon3.11/site-packages (from keras) (1.26.4)
Requirement already satisfied: rich in /Users/ameygaikwad/anaconda3/lib/pyth
on3.11/site-packages (from keras) (13.9.4)
Requirement already satisfied: namex in /Users/ameygaikwad/anaconda3/lib/pyt
hon3.11/site-packages (from keras) (0.1.0)
Requirement already satisfied: h5py in /Users/ameygaikwad/anaconda3/lib/pyth
on3.11/site-packages (from keras) (3.12.1)
Requirement already satisfied: optree in /Users/ameygaikwad/anaconda3/lib/py
thon3.11/site-packages (from keras) (0.18.0)
Requirement already satisfied: ml-dtypes in /Users/ameygaikwad/anaconda3/li
b/python3.11/site-packages (from keras) (0.5.4)
Requirement already satisfied: packaging in /Users/ameygaikwad/anaconda3/li
b/python3.11/site-packages (from keras) (24.2)
Requirement already satisfied: typing-extensions>=4.6.0 in /Users/ameygaikwa
d/anaconda3/lib/python3.11/site-packages (from optree->keras) (4.12.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /Users/ameygaikwad/a
naconda3/lib/python3.11/site-packages (from rich->keras) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /Users/ameygaikwa
d/anaconda3/lib/python3.11/site-packages (from rich->keras) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /Users/ameygaikwad/anaconda3/li
b/python3.11/site-packages (from markdown-it-py>=2.2.0->rich->keras) (0.1.0)
Note: you may need to restart the kernel to use updated packages.
```

In [25]:
```
pip install torch torchvision
```

```
Collecting torch
  Obtaining dependency information for torch from https://files.pythonhoste
d.org/packages/0f/8b/4b61d6e13f7108f36910df9ab4b58fd389cc2520d54d81b88660804
aad99/torch-2.10.0-2-cp311-none-macosx_11_0_arm64.whl.metadata
  Downloading torch-2.10.0-2-cp311-none-macosx_11_0_arm64.whl.metadata (31 k
B)
Collecting torchvision
  Obtaining dependency information for torchvision from https://files.python
hosted.org/packages/3e/be/c704bceaf11c4f6b19d64337a34a877fcdfe3bd68160a8c9ae
9bea4a35a3/torchvision-0.25.0-cp311-cp311-macosx_11_0_arm64.whl.metadata
  Downloading torchvision-0.25.0-cp311-cp311-macosx_11_0_arm64.whl.metadata
(5.4 kB)
Requirement already satisfied: filelock in /Users/ameygaikwad/anaconda3/lib/
python3.11/site-packages (from torch) (3.17.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /Users/ameygaikw
ad/anaconda3/lib/python3.11/site-packages (from torch) (4.12.2)
Requirement already satisfied: sympy>=1.13.3 in /Users/ameygaikwad/anaconda
3/lib/python3.11/site-packages (from torch) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /Users/ameygaikwad/anacond
a3/lib/python3.11/site-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in /Users/ameygaikwad/anaconda3/lib/py
thon3.11/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /Users/ameygaikwad/anaconda
3/lib/python3.11/site-packages (from torch) (2024.12.0)
Requirement already satisfied: numpy in /Users/ameygaikwad/anaconda3/lib/pyt
hon3.11/site-packages (from torchvision) (1.26.4)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /Users/ameygaikwad/a
naconda3/lib/python3.11/site-packages (from torchvision) (11.1.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /Users/ameygaikwad/anac
onda3/lib/python3.11/site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /Users/ameygaikwad/anacond
a3/lib/python3.11/site-packages (from jinja2->torch) (3.0.2)
Downloading torch-2.10.0-2-cp311-none-macosx_11_0_arm64.whl (79.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.4/79.4 MB 2.3 MB/s eta 0:00:0
000:0100:01m
Downloading torchvision-0.25.0-cp311-cp311-macosx_11_0_arm64.whl (1.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.9/1.9 MB 22.7 MB/s eta 0:00:00
a 0:00:01
Installing collected packages: torch, torchvision
Successfully installed torch-2.10.0 torchvision-0.25.0
Note: you may need to restart the kernel to use updated packages.
```

In [16]:
```
import os
os.environ["KERAS_BACKEND"] = "torch"  # we need to have this line set
before importing keras so that keras will use pytorch
import keras
import torch
```

To create a neural network, we first define the sequence of layers. Below is an example neural network with 3 neurons in the input layer, two hidden layers, with 5 neurons in each hidden layer, and one neuron in the output layer. We will use ReLU as our activation

function. Read more about the `Dense()` function to create fully connected layers here: https://keras.io/api/layers/core_layers/dense/

In [43]:
```
model = keras.Sequential([
    keras.Input((3,)), # input layer
    keras.layers.Dense(5, activation="relu"), # first hidden layer
    keras.layers.Dense(5, activation="relu"), # second hidden layer
    keras.layers.Dense(1)
])
```

We now need to compile the model in order to train it. The compile function takes in the optimizer, loss function, and metrics to evaluate during training. Read more about the compile function here: https://keras.io/api/models/model_training_apis/#compile-method

Here, we will use the Adam optimizer and mean squared error loss function.

In [44]:
```
# compile the model
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.02),
    loss="mse"
)
```

Now we can train the model. The fit function takes in the training data and the number of epochs to train for. Read more about the fit function here: https://keras.io/api/models/model_training_apis/#fit-method

In [45]:
```
## NOTE: if you run this cell multple times, it will continue to train from
the last time this cell is run. This means that if you train for 100
epochs, then rerun this cell, it will train for another 100 for a total of
200. If you want to retrain your model from scratch, you need to create the
model the compile it again before training!

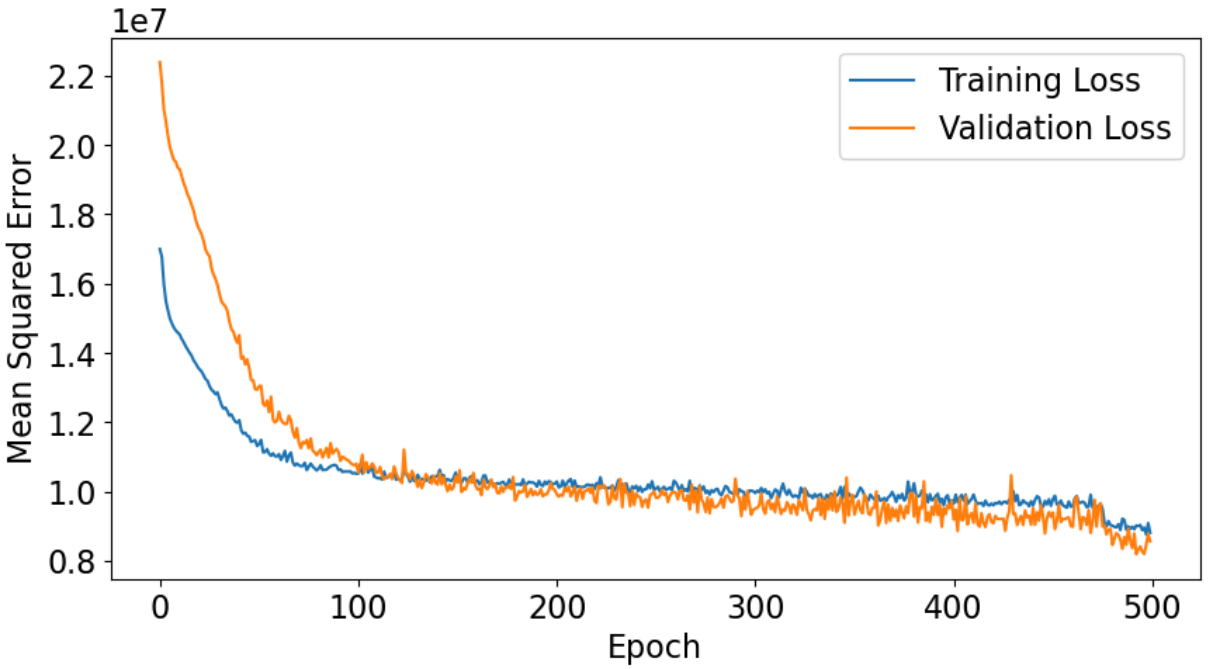
# this might make take a while to train depending on the number of epochs
history = model.fit(X_train, y_train, epochs=500,
    validation_split=0.2, # the model will internally take out random set
of 0.2 of the data to compute the validation loss at each step
    batch_size = 128,
    verbose=0 # set to 1 to display the loss and validation loss at every
epoch
)
```

In [46]:
```
# examine how the loss changes as a function of epochs for the training and
the internal validation splits

plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
```

```
plt.xlabel('Epoch')
plt.legend()
```

Out[46]: <matplotlib.legend.Legend at 0x298fa5390>



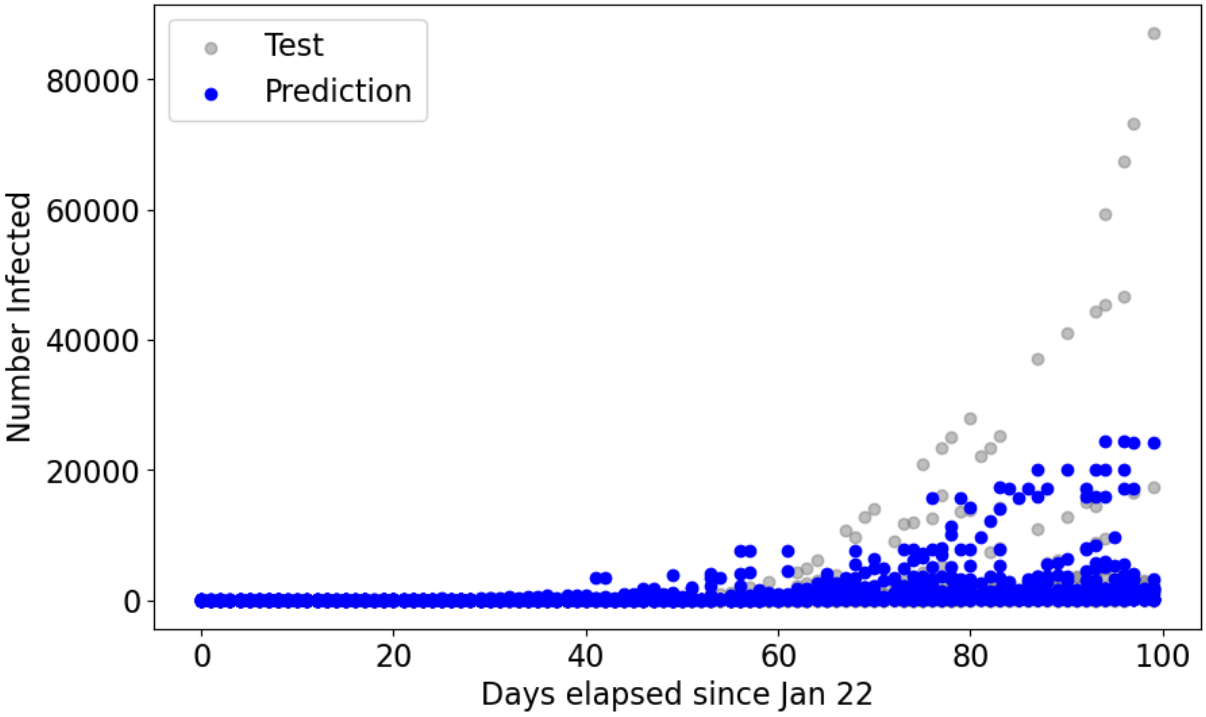
With 500 epochs and a learning rate of 0.02, we see the training and validation loss decrease systematically. The validation loss becomes less than the training loss with increasing epochs which is a positive sign !

In [47]:
```
# make model predictions using the test data
predictions = model.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions, label='Prediction', color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

Out[47]: Text(0, 0.5, 'Number Infected')



Saving the model

We can use keras to save the model so that we don't need to train it to make predictions for next it. It is also important to save your models as a record of the experiment.

In [48]:
```
# Save the entire model to a single file
model_save_file = 'covid19_initial_model.keras'
model.save(model_save_file)
print("Model saved to disk:" + model_save_file)
```

```
Model saved to disk:covid19_initial_model.keras
```

Loading the model

Once you have a .keras data file, you can load it into your program using the following code:

In [30]:
```
# This code is purposely commented out. Uncomment it if you want to load a
model

# Load the model exactly as it was
#reloaded_model = keras.models.load_model("covid19_initial_model.keras")

# Verify it works by making a prediction
#prediction = reloaded_model.predict(X_test)
```

In [50]:
```
# This code is purposely commented out. Uncomment it if you want to load a
model
```

```
# Load the model exactly as it was
reloaded_model = keras.models.load_model("covid19_initial_model.keras")

# Verify it works by making a prediction
prediction = reloaded_model.predict(X_test)
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

In [51]:
```
prediction.shape
```

Out[51]:
```
(1500, 1)
```

Question 2 (8 pts)

Run the cells above to create and train the initial model. Answer the following questions:

1. How many parameters are there in the model? How did you determine this? (1 pt)
2. Explain what is a batch size in training? (1 pt)
3. Explain what is the learning rate? (1 pt)
4. What is the difference between training loss and validation loss? (1 pt)
5. Describe the learning curve of the model. What do you notice about the training and validation curves? (2 pt)
6. Describe the final performance of the model. What fits well and what fits poorly? (2 pt)

1. 56 parameters in the system
2. The batch size in training the model above is 128.
3. The learning rate is 0.02.
4. The training loss is calculated on the training data, which is the set the model learns from. Its measured during training as the model updates its weights. Validation loss on the other hand is calculated on the validation data, which the model has not seen during training. Its used to check how well the model generalizes to new data. We want the model to learn from the training data and successfully generalize to new data like the validation set !
5. From the learning curve, we see that the difference between training and validation loss is small - specifically for the model above the training loss is lower than the validation loss initially. The validation loss becomes less than the training loss with increase in epochs which is good !
6. It fits the initial 'zero case' data well and it does poorly on the exponential increase in cases as the number of days elapsed increases. The trainign set might be oversampling on early data where we have slow growth, and hence is underesitmating the late data exponential blow up

In [49]:
```
print(model.count_params())
```

```
56
```

Step 6 - Fine Tune the Model

Neural networks can have many hyperparameters. Hyperparameters are parameters that are not learned from the data, but are set before training the model. These include the number of hidden layers, the number of nodes in each hidden layer, the activation function, the optimizer, the learning rate, and the number of epochs.

Question 3 (12 pts)

1. Create a new model with the same architecture as the initial model, but with a different number of epochs (e.g., 200 epochs instead of 100 epochs). Train the model and examine how the loss changes as a function of epochs for the training and the internal validation splits. Describe how the model performance changes as a function of epochs. Also describe the overall model performance after training. (3 pts)

2. Now, create a new model with the same architecture as the initial model, but with a different learning rate. Train the model and examine how the loss changes as a function of epochs for the training and the internal validation splits. Describe how sensitive the model is to learning rate. What happens if learning rate is too small or too large (vary by factors of 10 for this test). (3 pts)

3. Explore how the model performance changes as a function of the number of hidden layers and the number of hidden units per layer. (3 pts)

4. What did you find to be the best model architecture for this problem? Why? (3 pts)

I create a function below which should take in as inputs the number of hidden layers, hidden nodes in each layer (I am assuming same number of hidden nodes in each hidden layer) and the learning rate. The function returns the model to me. The activation funcion I use is RELU.

In [50]:
```
def create_model(hidden_layers,hidden_nodes,learning_rate):
    model=keras.Sequential()
    model.add(keras.Input(shape=(3,)))

    for _ in range(hidden_layers):
        model.add(keras.layers.Dense(hidden_nodes,activation="relu"))

    model.add(keras.layers.Dense(1))
    model.compile(
```

```
        optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
        loss="mse"
    )
    return model
```

3.1

Change the number of epochs to 1500. Keep the learning rate, hidden layers and number of hidden nodes the same as before - 0.02, 2 and 5, respectively.

In [51]:
```
# test the function created above with the same inputs as the model above
but now with 1500 epochs
model_test=create_model(2,5,0.02)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2,
    batch_size = 128,
    verbose=0 #
)

# examine how the loss changes as a function of epochs for the training and
the internal validation splits
```

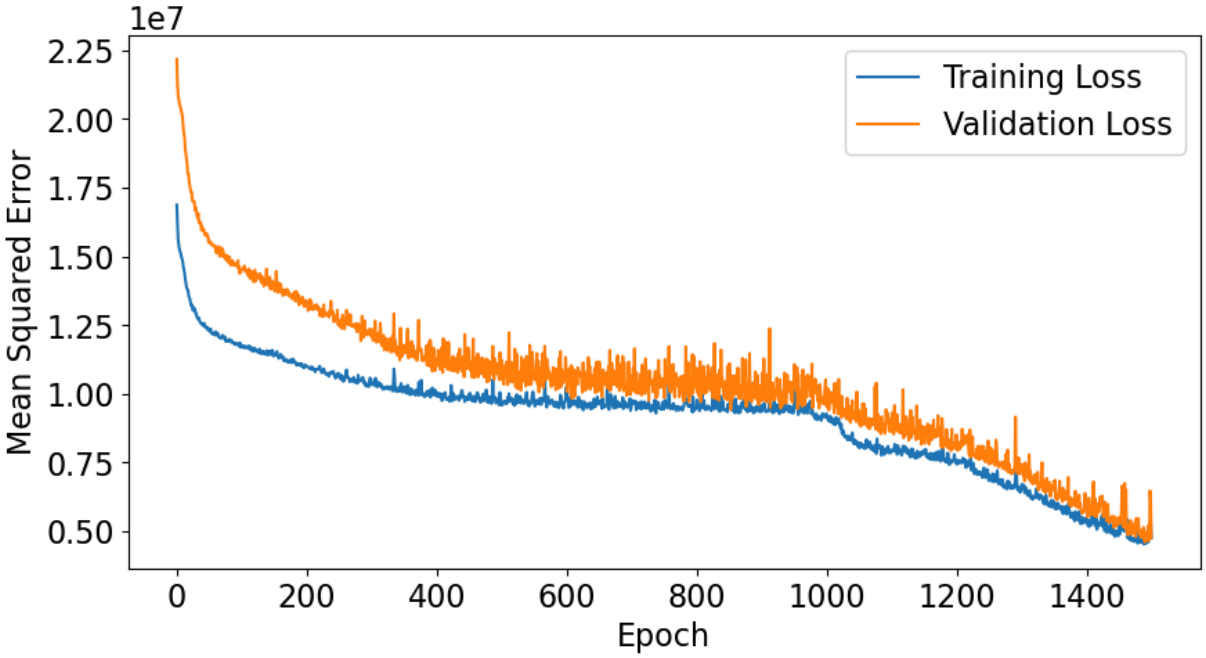
In [52]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
plt.xlabel('Epoch')
plt.legend()

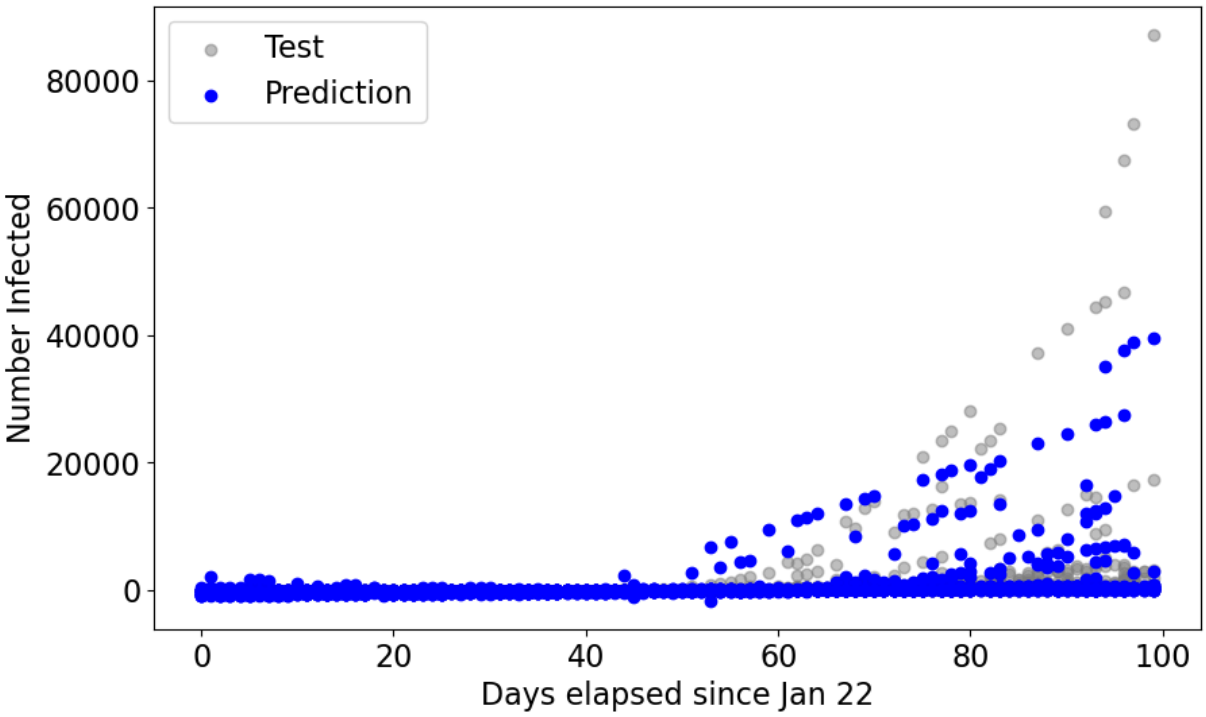
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 968us/step
```

Out[52]:
```
Text(0, 0.5, 'Number Infected')
```





In [53]:
```
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error

r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.7327
RMSE: 2586.66
```

We use R^2 score to quantify how well our model is working. In a perfect model (perfect predicitons), $R^2 = 1$. So we would like the R^2 to be as close to 1 as possible. A very poor model will have an R^2 score close to 0.

TRAINING AND VALIDATION LOSSES : Note that the training and validation loss almost thermalize with increasing number of epochs. These losses drop and with increasing number of epochs the model is trained so well that the validation loss is very close to the training loss ! This essentially implies that the model generalizes very well to new data.

PREDICTIONS : In line with the expectations following the low training and more importantly the validation loss, we see the precitive power of our model improving substantially ! Before (500 epochs) the model was not predicting the exponential blow up in the cases for late data. But now after training for 1500 epochs, the model indeed predicts the exponential blow up much better !

3.2

VARIATION WITH DIFFERENT LEARNING RATES

We consider learning rates close to 0.02 and one with a very high learning rate of 0.1. The model with this high learning rate behaves very badly ! It does not generalize well to validation data and has a negative R^2 score, which is maximally bad ! We also a pick a very small learning rate of 0.002 - we see that the losses go down but the 1500 epochs is probably too less for the model to train and generalize well. To see which learning rate could be optimal, I play around with the learning rate close to 0.02 which was working well above. I find 0.025 works pretty well returning an R^2 score of 0.87.

-- LEARNING RATE OF 0.1

In [54]:
```
model_test=create_model(2,5,0.1)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2, # the model will internally take out random set
of 0.2 of the data to compute the validation loss at each step
    batch_size = 128,
    verbose=0 # set to 1 to display the loss and validation loss at every
epoch
)
```

In [55]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
```

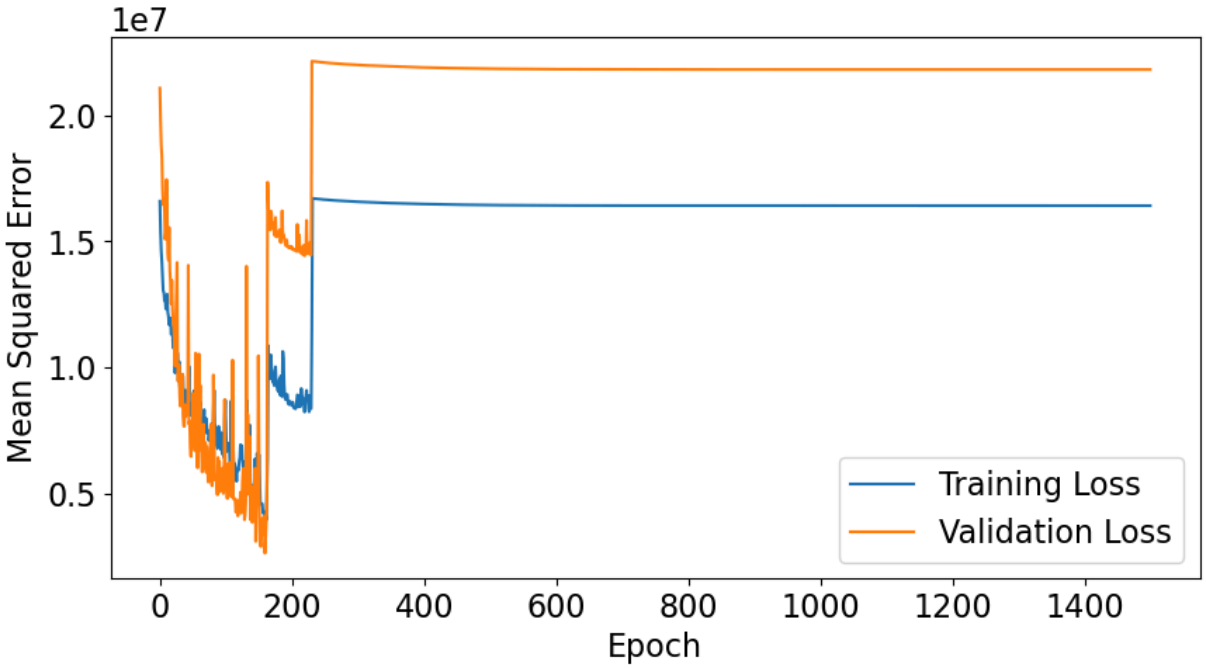
```
plt.xlabel('Epoch')
plt.legend()

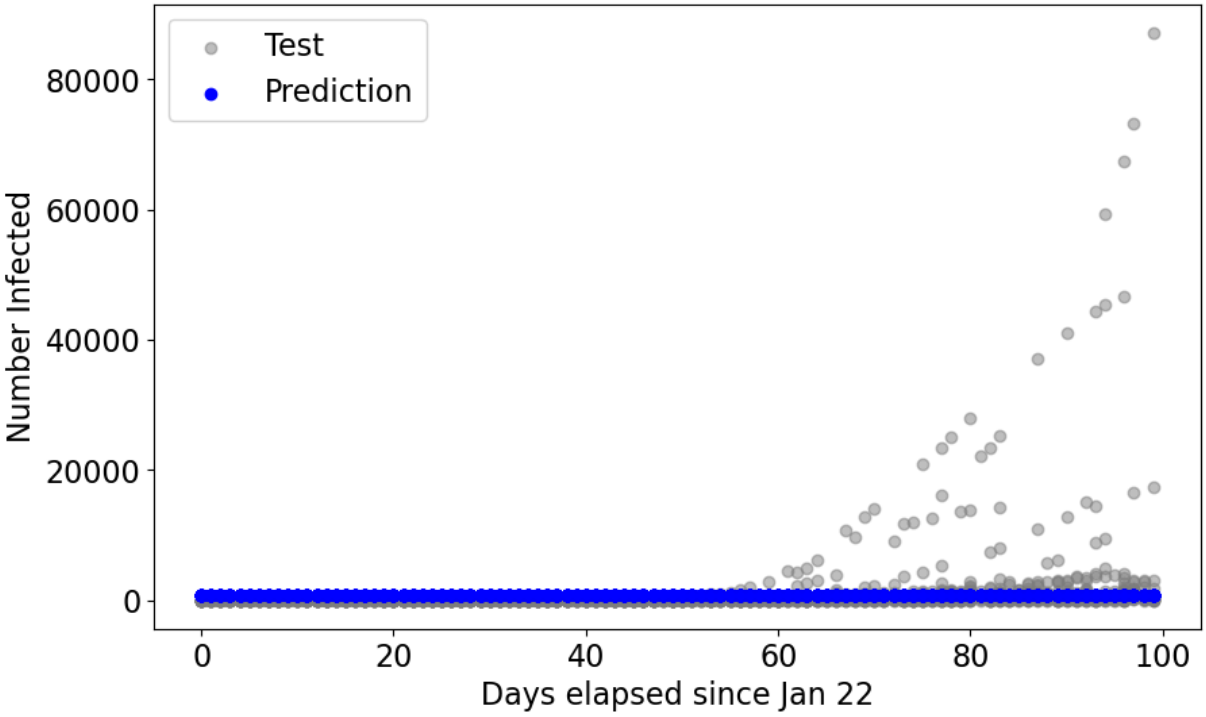
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 839us/step
```

Out[55]: Text(0, 0.5, 'Number Infected')





In [56]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: -0.0003
RMSE: 5003.89
```

The model performs very badly - this can be seen from the behaviour of the trianing and validation losses and also from the predictions graph. The learning rate of 0.1 is too high ! Also note that in this case the R^2 score is negative, so the model is worse than just predocting the average.

-- LEARNING RATE OF 0.01

In [57]:
```
model_test=create_model(2,5,0.01)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2, # the model will internally take out random set
of 0.2 of the data to compute the validation loss at each step
    batch_size = 128,
    verbose=0 # set to 1 to display the loss and validation loss at every
epoch
)
```

In [58]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
```

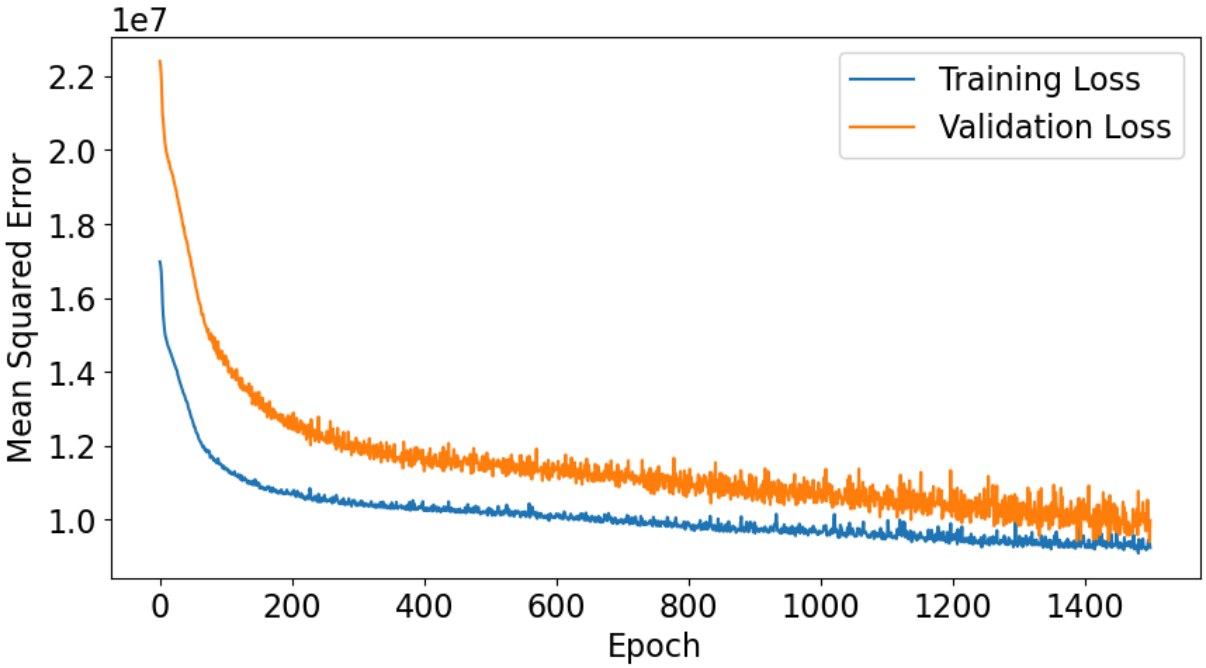
```
plt.xlabel('Epoch')
plt.legend()

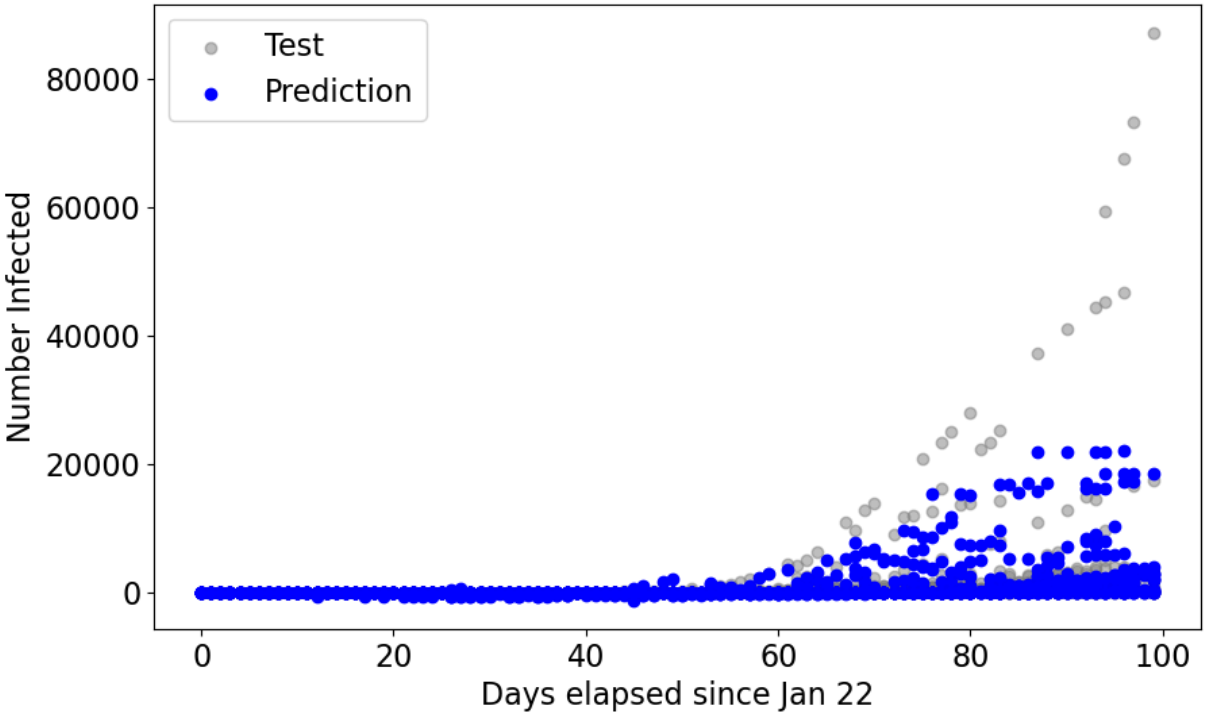
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 943us/step
```

Out[58]: Text(0, 0.5, 'Number Infected')





In [59]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.4154
RMSE: 3825.38
```

This learning rate of 0.01 is not terrible but it is worse than 0.02. The MSE error is higher than the model with 0.02 as learning rate. From the predictions graph, consequently we can also see that the predictive power of this model is not better than the model w 0.02 learning rate. Thus the rate of 0.01 is too low to generalize on data well. Additionally, we can compare the R^2 score - with a learning rate of 0.01, $R^2 = 0.41$, as opposed to 0.7 for the model with a learning rate of 0.02.

-- LEARNING RATE OF 0.025

In [62]:
```
model_test=create_model(2,5,0.025)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2, # the model will internally take out random set
of 0.2 of the data to compute the validation loss at each step
    batch_size = 128,
    verbose=0 # set to 1 to display the loss and validation loss at every
epoch
)
```

In [63]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
```

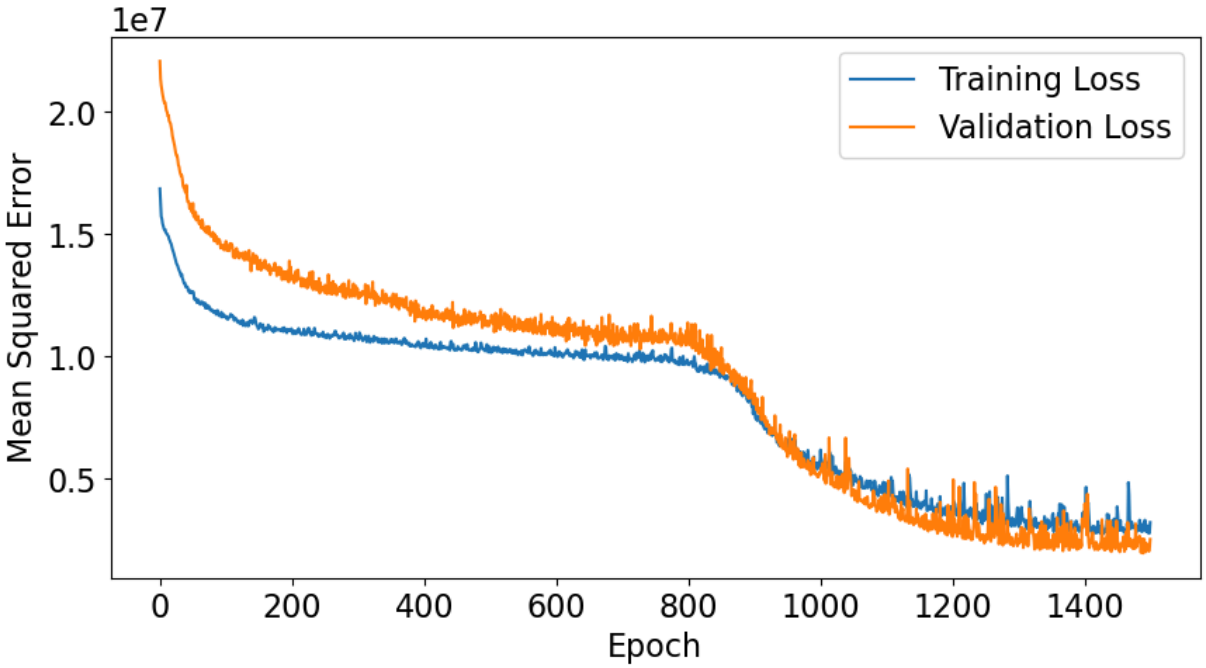
```
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
plt.xlabel('Epoch')
plt.legend()

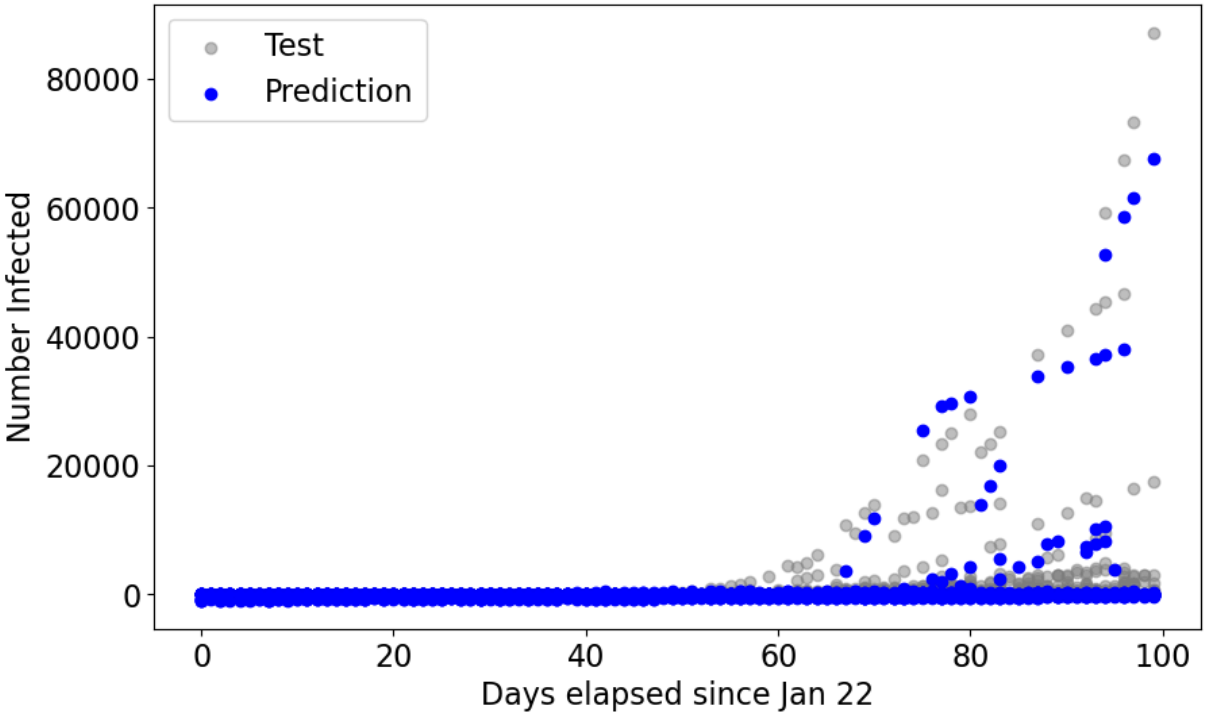
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 946us/step
```

Out[63]: Text(0, 0.5, 'Number Infected')





In [64]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.8719
RMSE: 1790.83
```

This learning rate of 0.025 is also good - the training and validation losses are low w validation going below training w increase in epochs. The prediction are also very nice and the model seems to be generalizing well to the exponential blow up. The R^2 score is also very high 0.87 !!

-- LEARNING RATE OF 0.002

In [83]:
```
model_test=create_model(2,5,0.002)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2, # the model will internally take out random set
of 0.2 of the data to compute the validation loss at each step
    batch_size = 128,
    verbose=0 # set to 1 to display the loss and validation loss at every
epoch
)
```

In [84]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
```

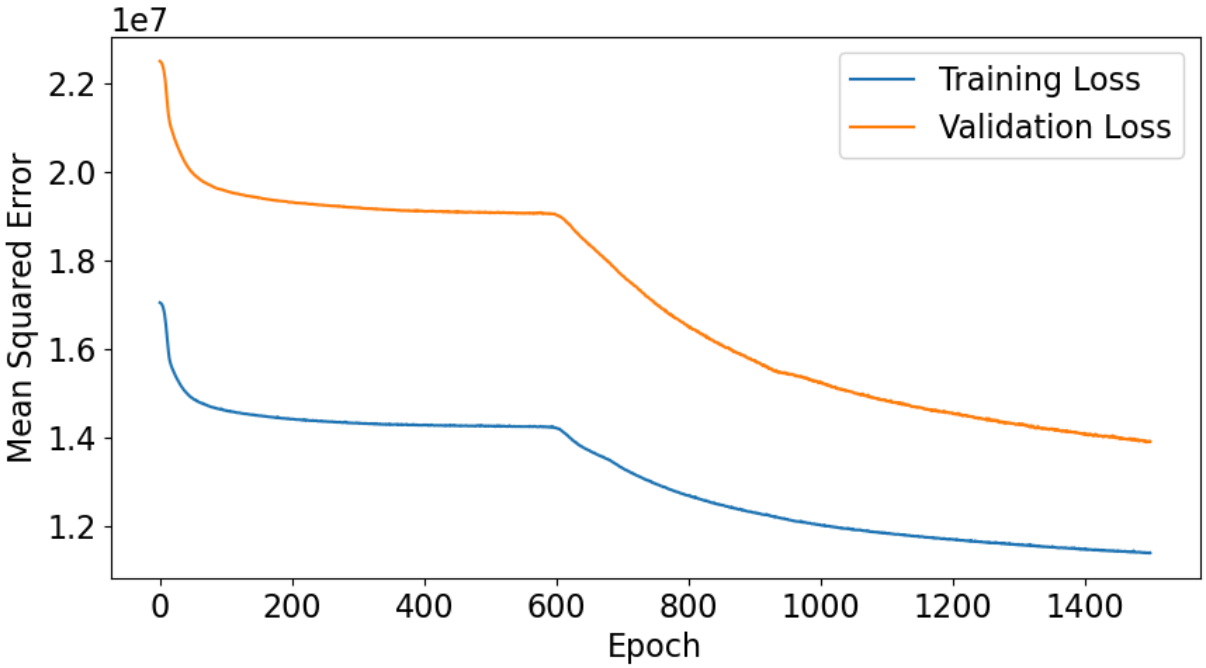
```
plt.xlabel('Epoch')
plt.legend()

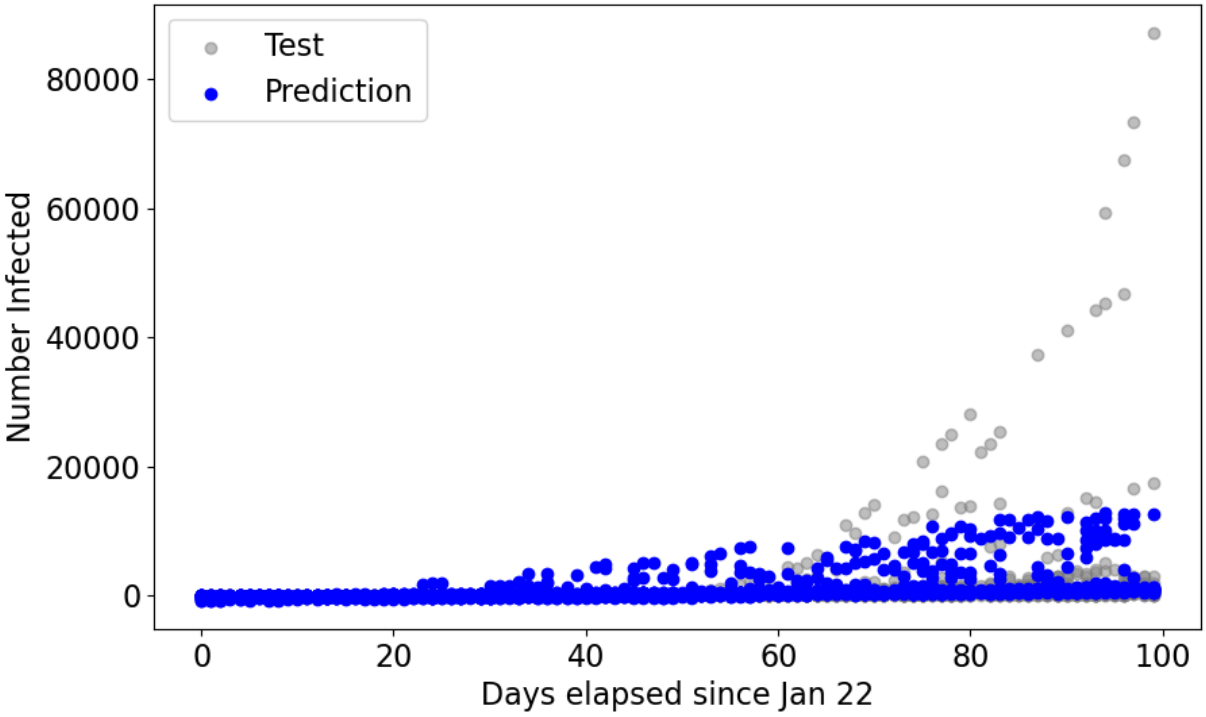
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

Out[84]: Text(0, 0.5, 'Number Infected')





In [85]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.2971
RMSE: 4194.39
```

The learning rate is too small for the model to generalize well and returns an R^2 score of 0.29 which is very small !

3.3

CHANGE NUMBER OF HIDDEN LAYERS AND HIDDEN NODES

NUMBER OF HIDDEN LAYERS : 3 AND THE LEARNING RATE (0.003). NUMBER OF HIDDEN NODES PER LAYER 5

In [65]:
```
model_test=create_model(3,5,0.003)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2,
    batch_size = 128,
```

```
    verbose=0
)
```

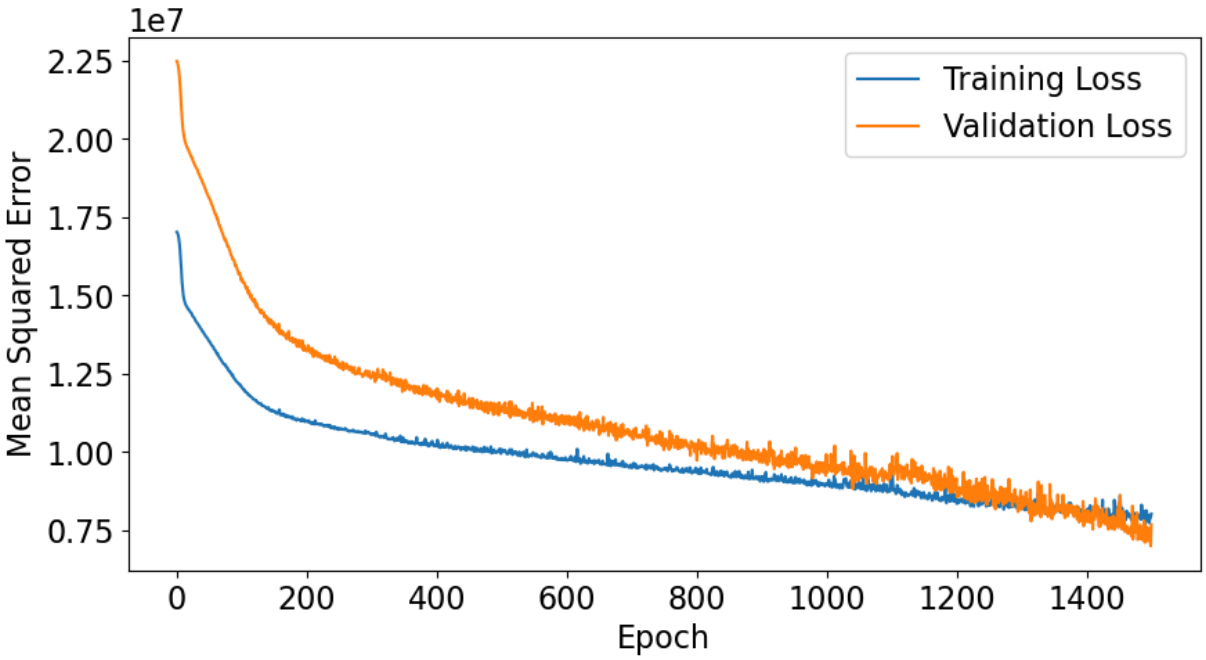
In [66]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
plt.xlabel('Epoch')
plt.legend()

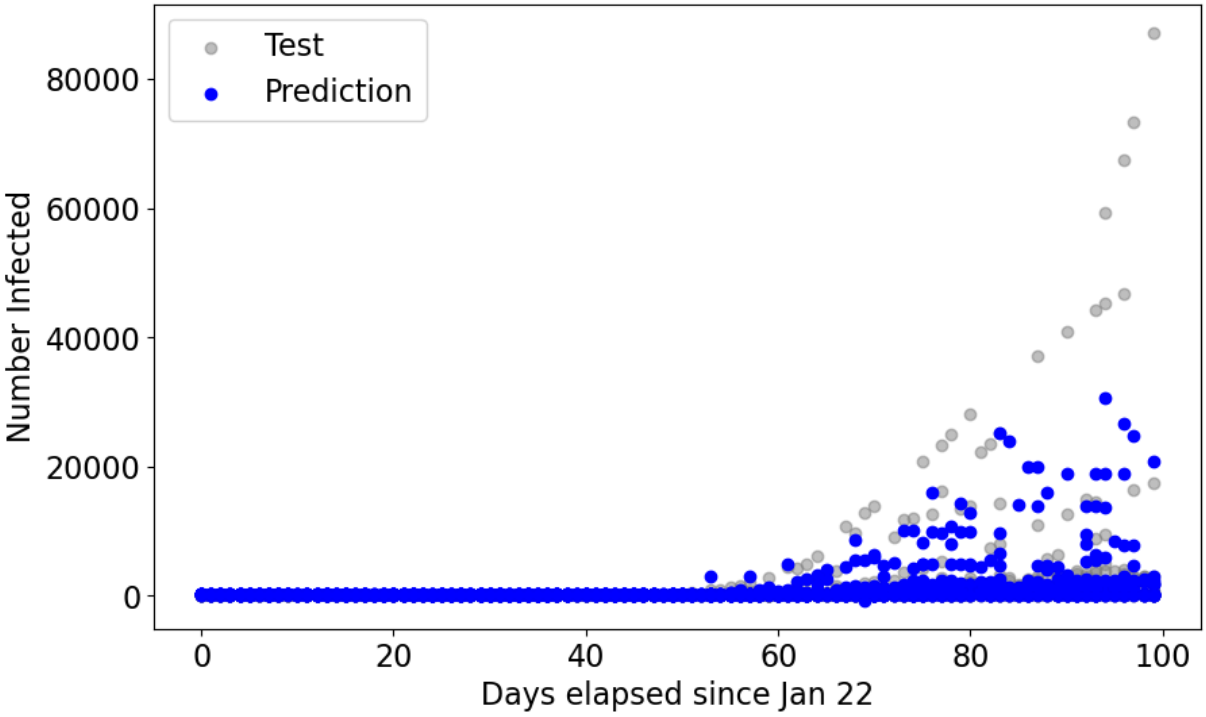
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

Out[66]:
```
Text(0, 0.5, 'Number Infected')
```





In [67]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.4720
RMSE: 3635.43
```

With an R^2 score of 0.47, this model is not the most optimal. Above we have examples which perform better than this.

NUMBER OF HIDDEN LAYERS : 4 AND THE LEARNING RATE (0.003). NUMBER OF HIDDEN NODES PER LAYER 5

In [68]:
```
model_test=create_model(4,5,0.003)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2,
    batch_size = 128,
    verbose=0
)
```

In [69]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
```

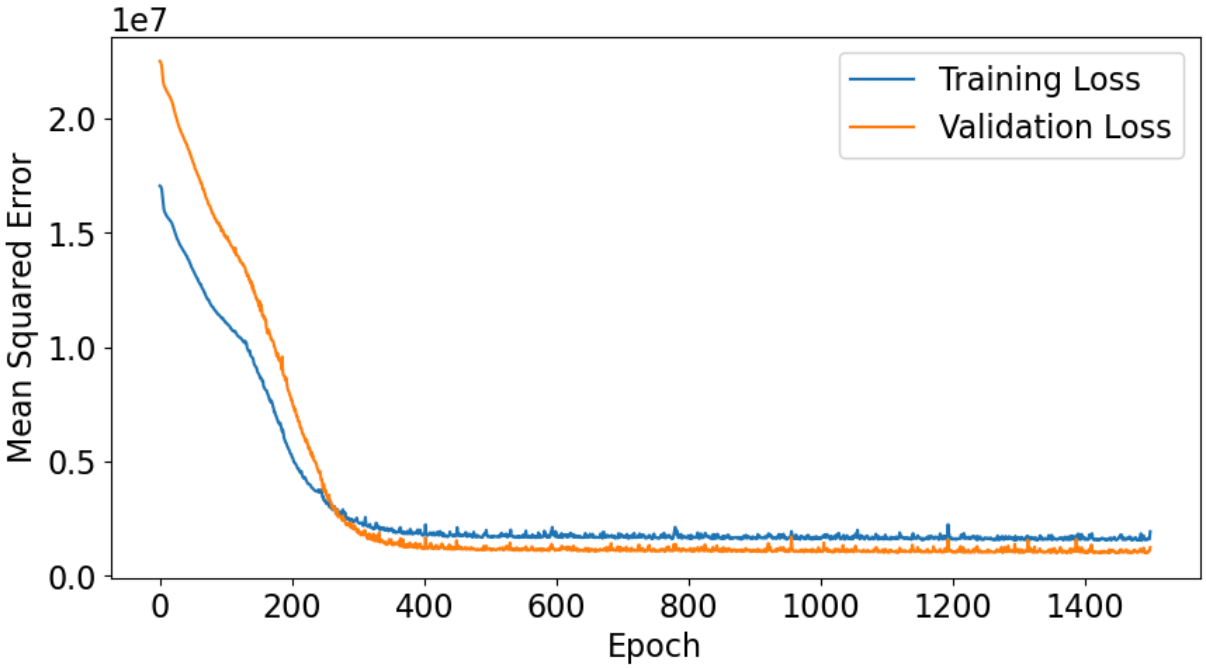
```
plt.xlabel('Epoch')
plt.legend()

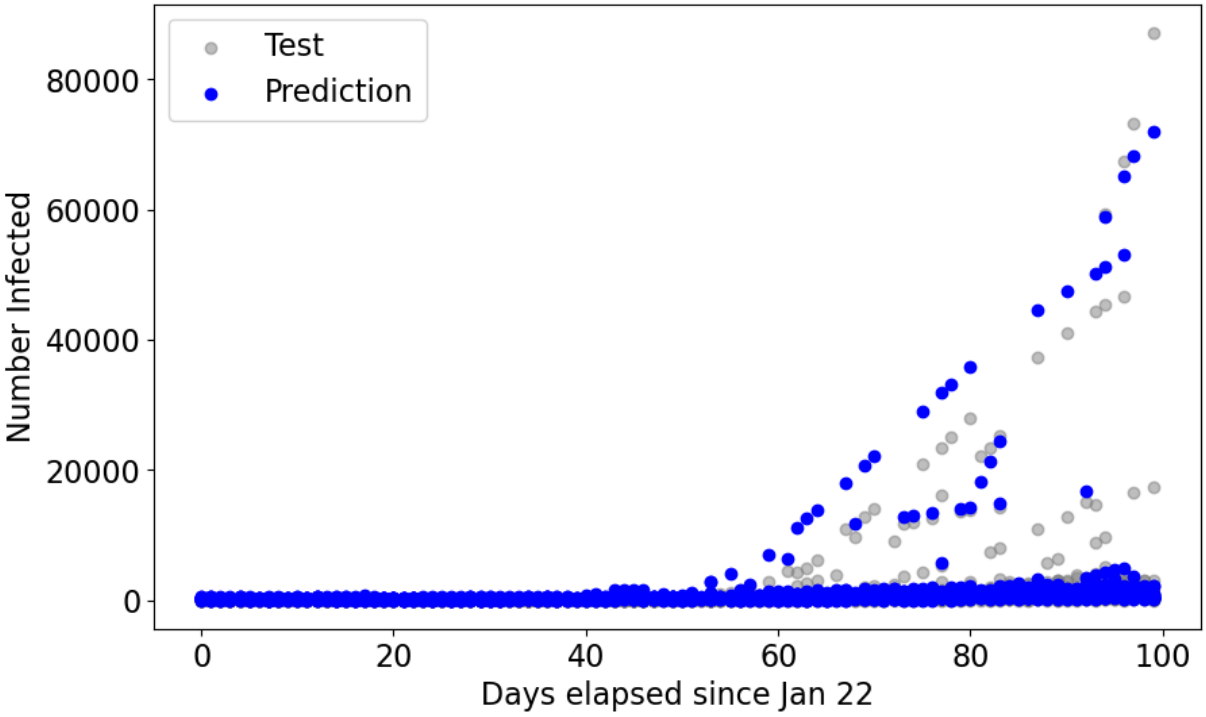
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

Out[69]: Text(0, 0.5, 'Number Infected')





In [70]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.9188
RMSE: 1426.00
```

This architecture does very well and returns an R^2 score of 0.91 !!

NUMBER OF HIDDEN LAYERS : 5 AND THE LEARNING RATE (0.003). NUMBER OF HIDDEN NODES PER LAYER 5

In [71]:
```
model_test=create_model(5,5,0.003)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2,
    batch_size = 128,
    verbose=0
)
```

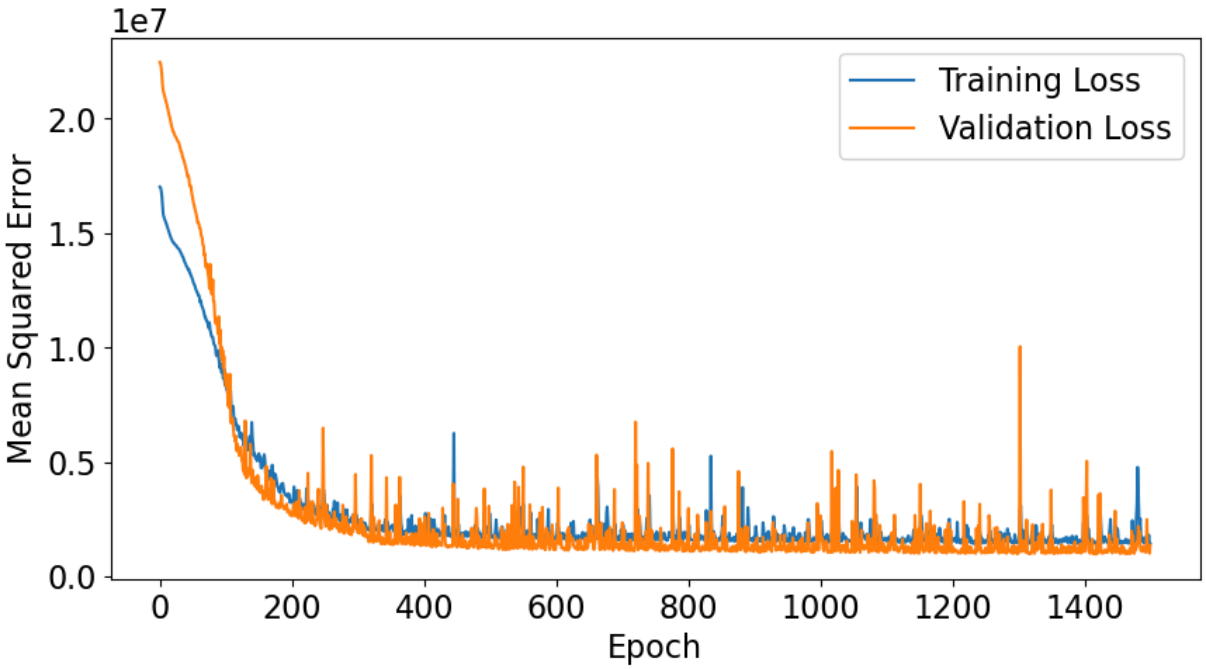
In [72]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
plt.xlabel('Epoch')
plt.legend()
```

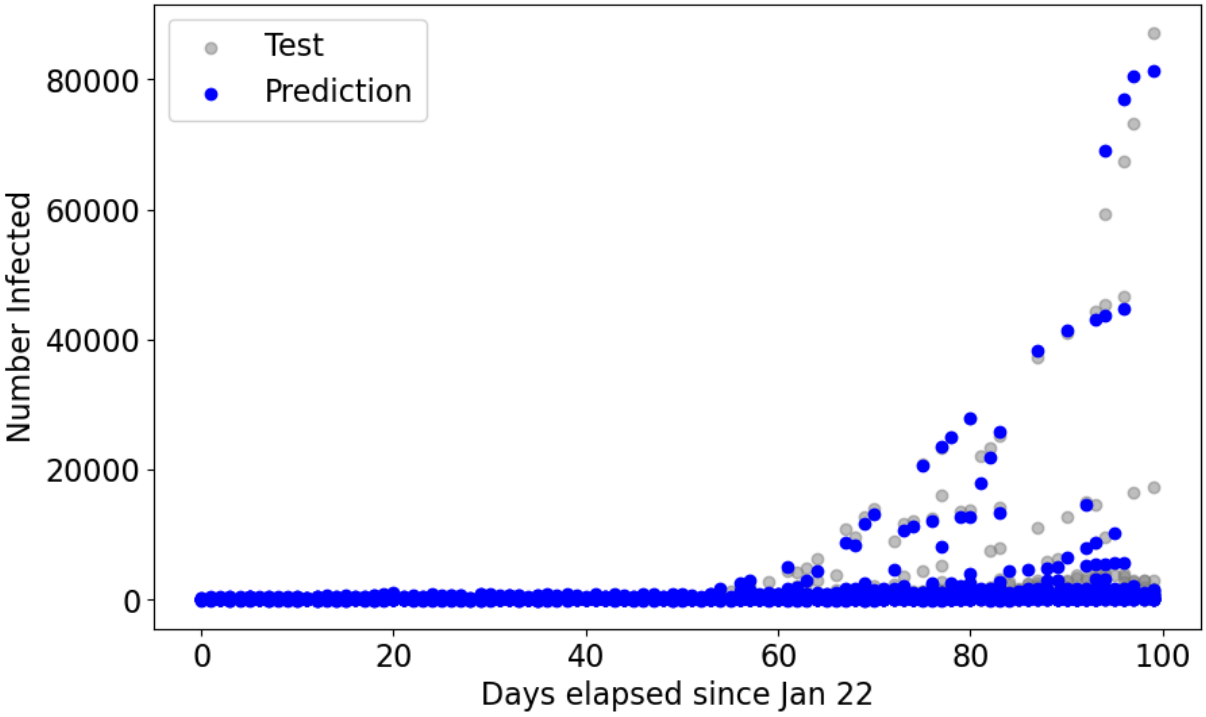
```
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

Out[72]: Text(0, 0.5, 'Number Infected')





In [73]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.9417
RMSE: 1207.79
```

THe best model so far with R^2 of 0.94 !

NUMBER OF HIDDEN LAYERS : 4 AND THE LEARNING RATE (0.003). NUMBER OF HIDDEN NODES PER LAYER 6

In [74]:
```
model_test=create_model(4,6,0.003)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2,
    batch_size = 128,
    verbose=0
)
```

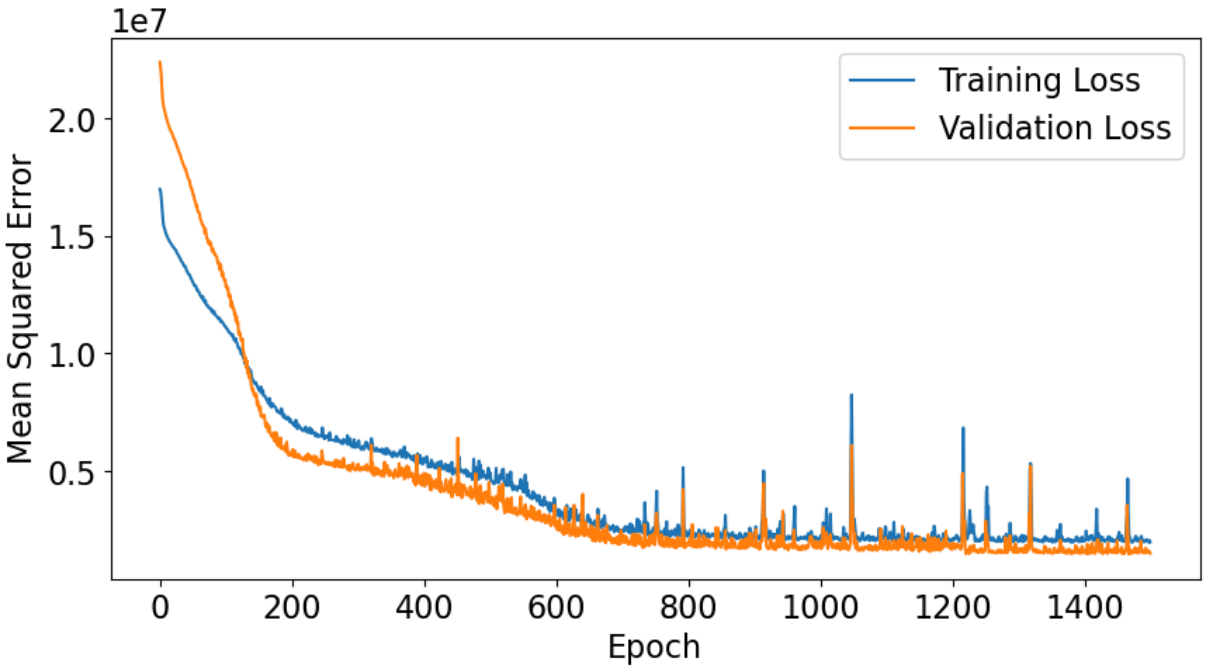
In [75]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
plt.xlabel('Epoch')
plt.legend()
```

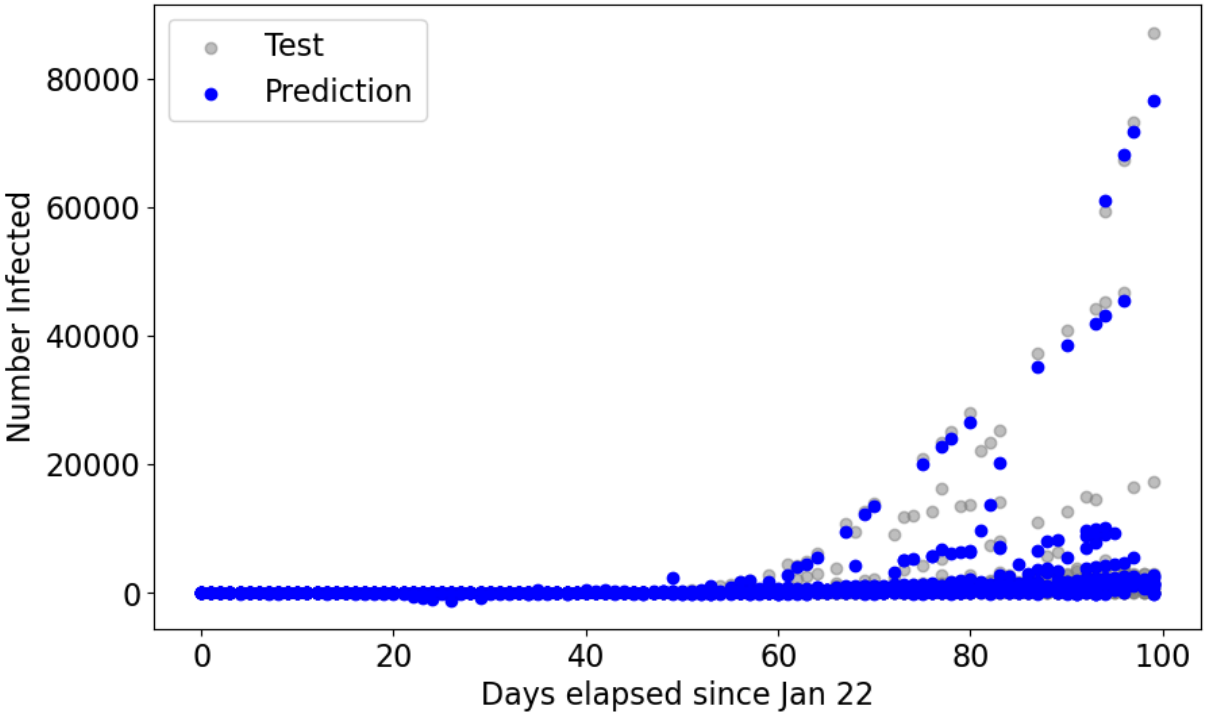
```
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

Out[75]: Text(0, 0.5, 'Number Infected')





In [76]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.9242
RMSE: 1377.27
```

Performs better than the model with the same parameters but less number of hidden nodes per layer and returns R^2 of 0.92 ! !

NUMBER OF HIDDEN LAYERS : 4 AND THE LEARNING RATE (0.003). NUMBER OF HIDDEN NODES PER LAYER 10

In [77]:
```
model_test=create_model(4,10,0.003)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2,
    batch_size = 128,
    verbose=0
)
```

In [78]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
```

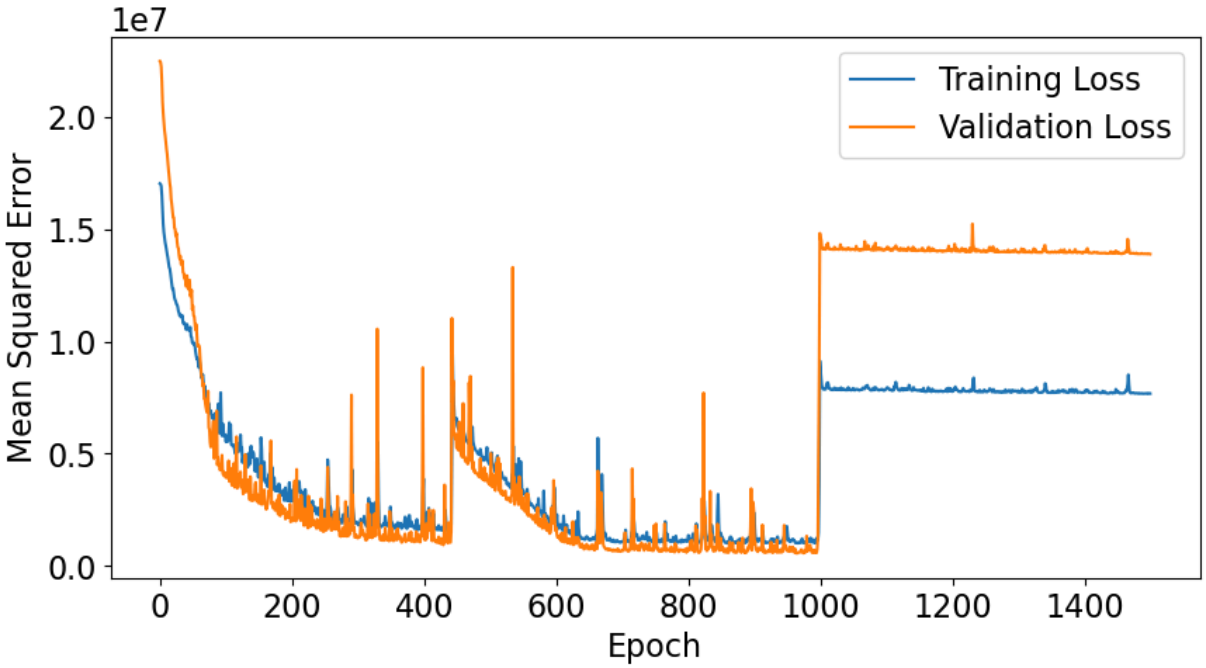
```
plt.xlabel('Epoch')
plt.legend()

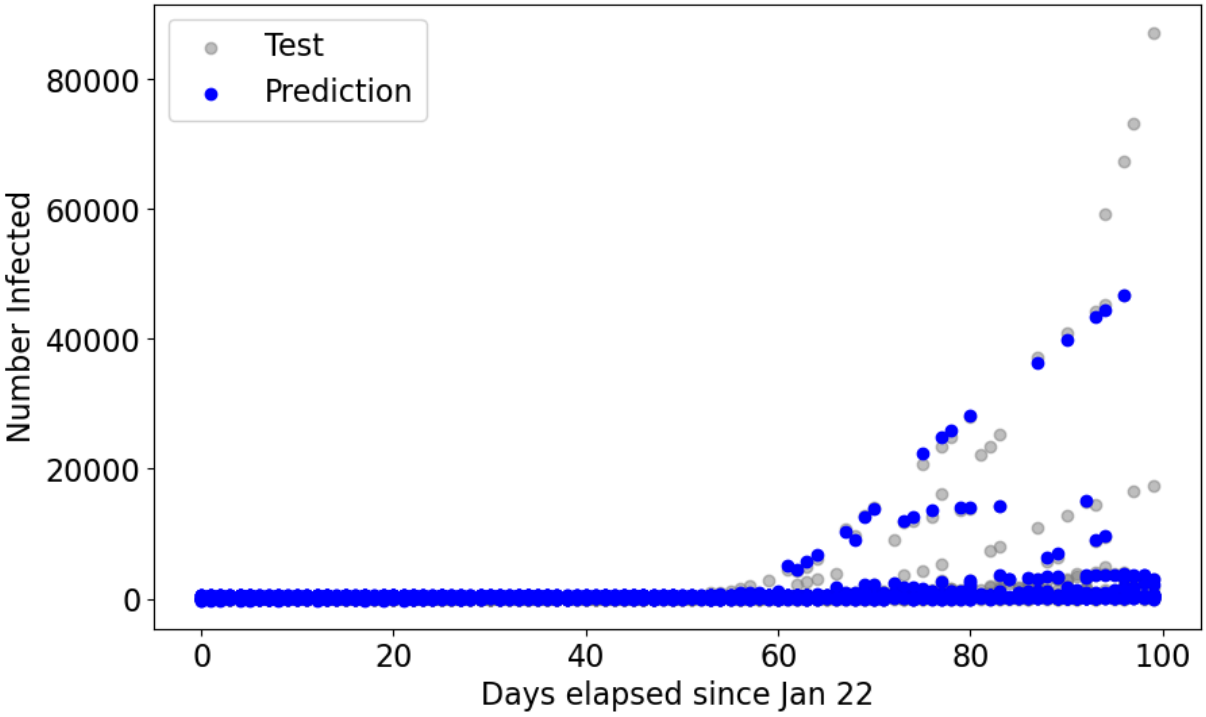
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

Out[78]: Text(0, 0.5, 'Number Infected')





In [79]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.3583
RMSE: 4007.58
```

Performs badly !!! Randomly changing the number of hidden nodes, while keeping everything else the same does not always improve the model ! More nodes can make the performance worse. It can lead to overfitting, since mode nodes imply more paramters and hence more capacity to memorie. The model might be fitting training true patterns instead of true patterns.

NUMBER OF HIDDEN LAYERS : 5 AND THE LEARNING RATE 0.002 (This is a lower rate than the one used for other examples in this section. I have done this to compensate for the increase in the number of hidden layers and number of hidden nodes per layer). NUMBER OF HIDDEN NODES PER LAYER 15

In [80]:
```
model_test=create_model(5,15,0.002)
history = model_test.fit(X_train, y_train, epochs=1500,
    validation_split=0.2,
    batch_size = 128,
```

```
    verbose=0
)
```

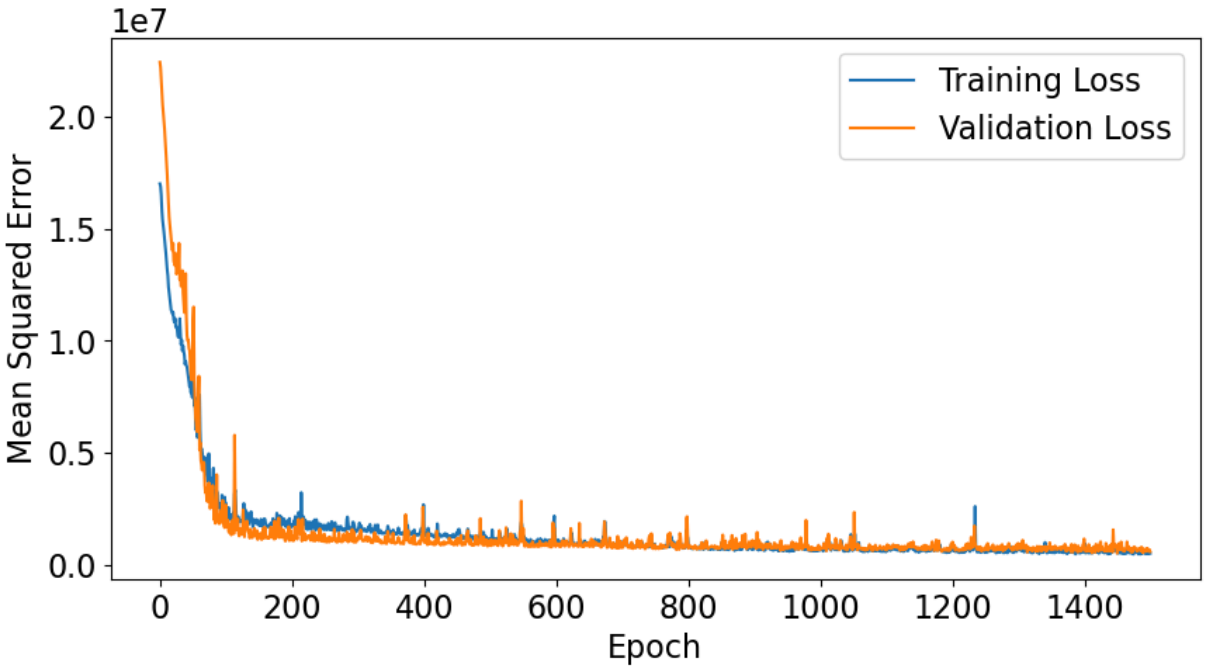
In [81]:
```
plt.figure(figsize=(10, 5))
# Plot training loss
plt.plot(history.history['loss'], label='Training Loss', color='#1f77b4')
# Plot validation loss
plt.plot(history.history['val_loss'], label='Validation Loss',
color='#ff7f0e')
plt.ylabel('Mean Squared Error')
plt.xlabel('Epoch')
plt.legend()

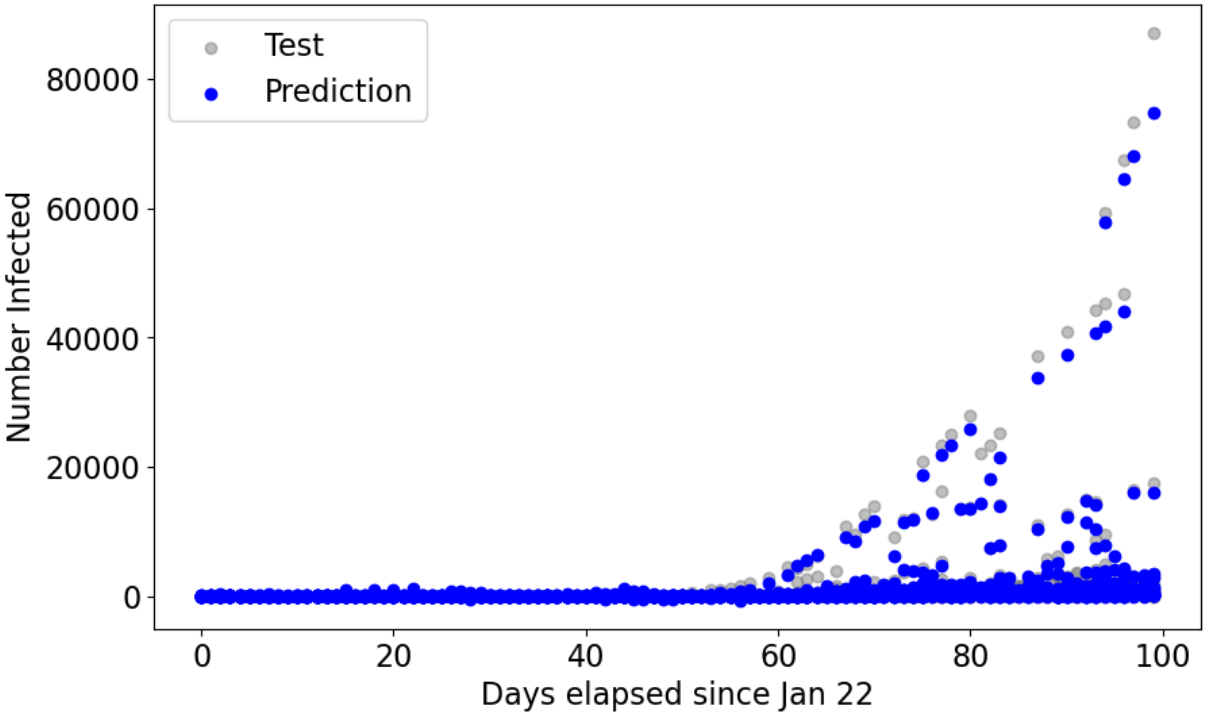
# make model predictions using the test data
predictions_test = model_test.predict(X_test)

# plot the model predictions
plt.figure(figsize=(10,6))
plt.scatter(X_test[:,2], y_test, label='Test', color='gray', alpha=0.5)
plt.scatter(X_test[:,2], predictions_test, label='Prediction',
color='blue')
plt.legend()
plt.xlabel('Days elapsed since Jan 22')
plt.ylabel('Number Infected')
```

47/47 ━━━━━━━━━━━━━━━━━━━━ **0s** 2ms/step

Out[81]: Text(0, 0.5, 'Number Infected')





In [82]:
```
r2 = r2_score(y_test, predictions_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions_test))

print(f"R² Score: {r2:.4f}")
print(f"RMSE: {rmse:.2f}")
```

```
R² Score: 0.9707
RMSE: 856.12
```

This is the best model constructed so far !! Generically, increasing the number of hidden layers and hidden nodes increases the performance of the model. The key operative word is generically - we clearly presented an exampele above where we naively just pump up the number of hidden nodes. This leads to a bad model architecture. We try to correct for this naivety by simultaneoulsy also dialing down the learning rate in the final case example and this ends up working beautifully !

Step 7 - Present your solution

Question 4 (6 pts)

1. Describe examples where the model worked well and examples where it failed. Hypothesize why this happened. (3 pts)
2. What are the implications of this model for our original goal of COVID19 prediction? (3 pts)

As explained in the detailed comments above, there are clear examples where the model architecture failed :

- If the learning rate is too high, the model performs worse - the training and validation losses fluctuate and do not thermalize with increasing epochs.
- If the learning rate is too slow , the model does not generalize quickIY and we need to train the model over a large number of epochs. In the example above with learning rate 0.002 and 2 hidden layers, we see that the losses do not thermalize over 1500 epochs.
- The learning rate chosen delicately depends on the number of hidden layers and the number of nodes chosen in each hidden layer. What might be an optimal learning rate for a given choice of hidden layers might not be optimal if we increase the hidden layers. This is clear from when we increase the number of layers from just 2 to 3.
- Generically, we expect that increasing the number of hidden layers and nodes increases the model performance, and we indeed see this over the examples displayed above. There are specific cases, also highlighted above, for instance, when we consider 10 nodes and 4 layers, where just naively increasing the hidden parameters leads to a much worse model architecture and the model performs badly. This is because more layers and nodes gives us more number of parameters to play around with which increases the risk of overfitting.
- The best model I construct above is with 5 hidden layers, 15 hiddden nodes and a learning rate of 0.002.

With an R^2 score of 0.974, my best model demonstrates strong predictive capability. The implications and limitations to consider are the following :

- The high score indicates the model captures 97% os the variance in the test data suggesting it learned the underlying pattersn of disease spread effectively within the time period and geographic locations represented by the chosen data.
- The model successfully captures complex non linear relationships betweeen time, location and case counts, which suggests neural networks can model epidemic dynamics better than simple linear models.
- Given this data, for near temr prediction, and more importantly, where the conditions remain similar to the training data, we could have used this model to provide estimates for public health response !
- There are clear limitations to the model. The model is trained on data from early 2020. It does not account for the new variants that emerged later, like Delta or Omicrrons. It of course does not account for population immunity changes, eg. herd immunity !
- Our model does a pretty food job at predicting cases in the exponential blow up pahse , but it cannot extrapolate well arbitrarily - there are clear instances in the predictions graph, where we see the model perform badly. Neural networks trained

on predominantly early stage data cannot reliably extrapolate to these exponential phases.

- We miss many variables which will be very useful to make our model better, eg. population density (through the examples of China and India, I display how different these countries were affected by COVID), healthcare infrastructure, government regulations, etc !
- We trained only on 50 locations. This might not be generalize to new locations with much different characteristics, climate or population behaviours.
- Potential data quality issues - underreporting of cases and reporting delays can drastically change modeling patterns !

This very basic model can provide a baseline to give estimates for covid surge. It can help identify trneds and hotspots in the training period. I do not think it can predict futrture waves caused by new variansts nad generalize beyond the training time period reliably, since epidemic spread is extremely sensitive to small changes to what we consider as input data !